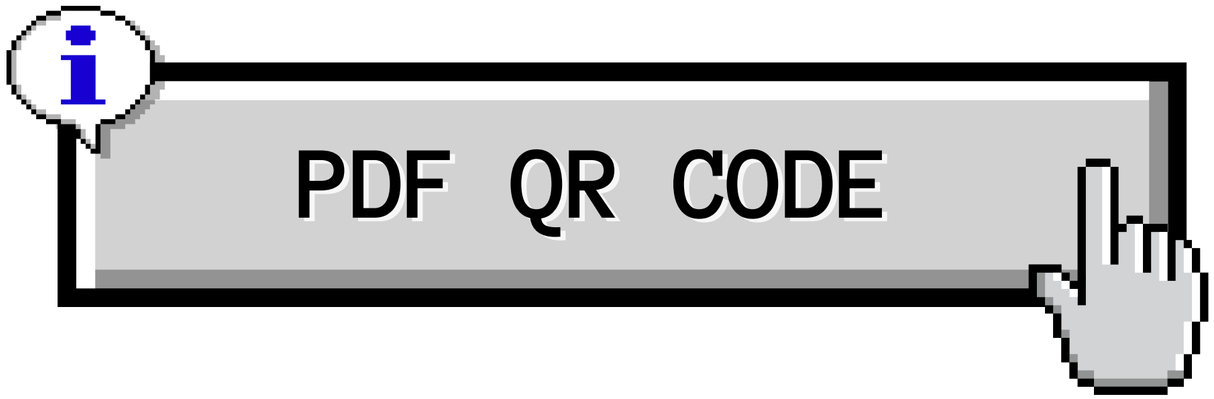
GoF Gonna Give You Up...

Home Content Live Demo Conclusion



GoF Gonna Let You Down...

Home Content Live Demo Conclusion



GoF Gonna Run Around...

Home Content Live Demo Conclusion

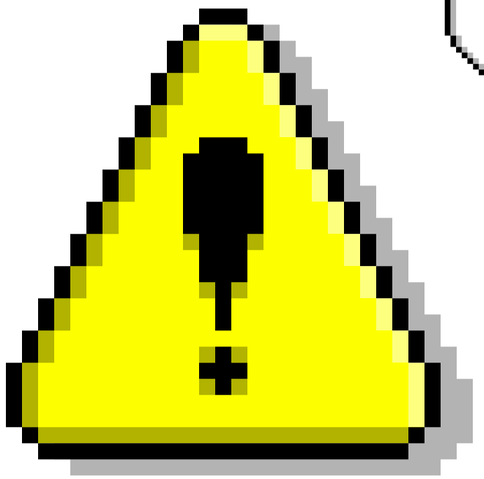
Introduction to

TIVIDY

WORKFLOW ENGINE

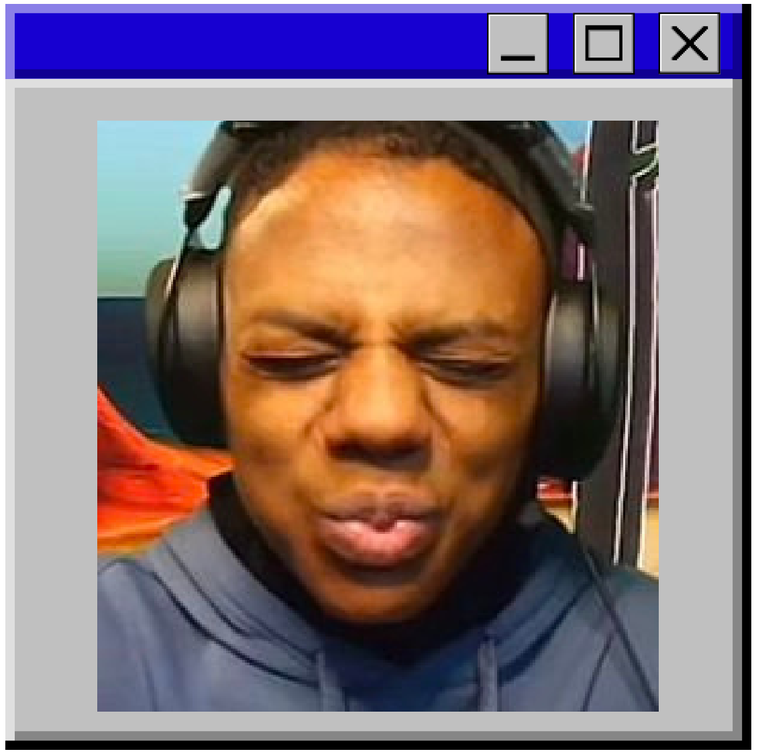


Start



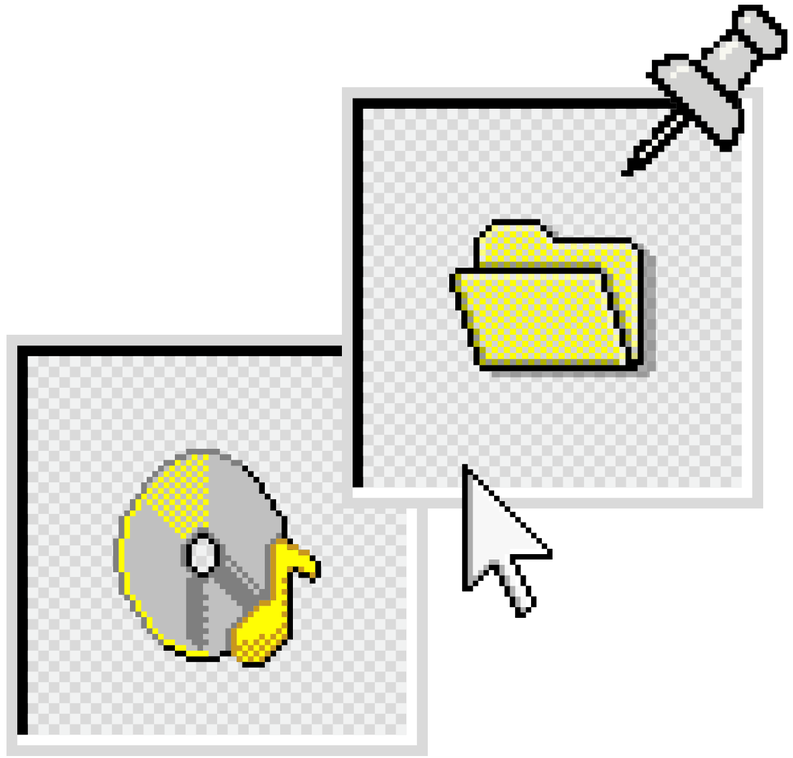
THIS IS A SIMULATION FOR A WORKFLOW MANAGEMENT SYSTEN

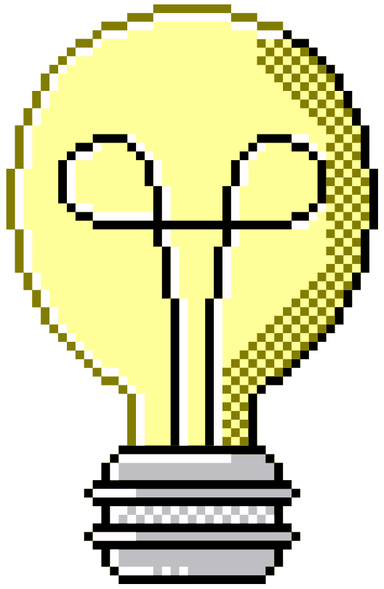
The generic and specific scenario systems do not reflect actual industry workflows.



CONTENT

- **Project Scope**
- **Backend**
  - UML Class Diagram
  - Design Pattern Uses
    - Why and How
- **GitHub**
  - GitHub Actions
- **Doxygen and Docker**
- **Tests**
- **Scenarios**
- **Frontend**
  - Live Demo





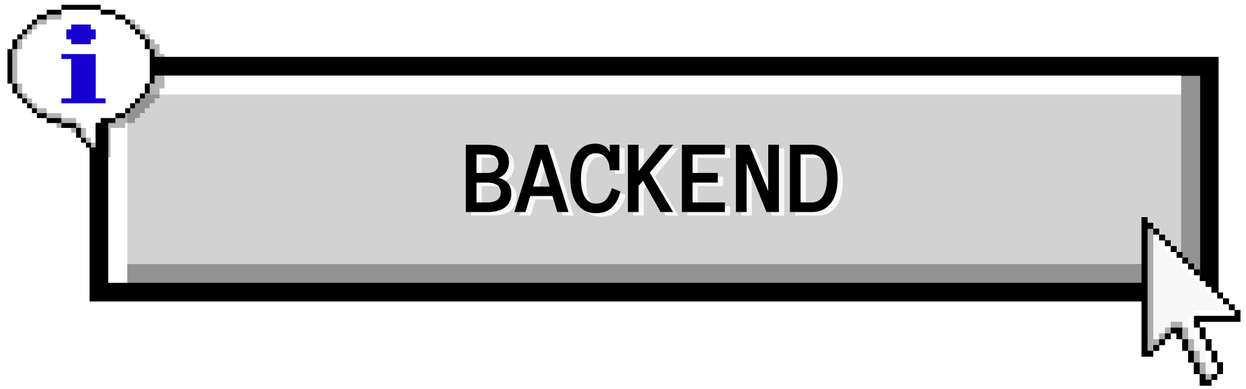
PROJECT SCOPE

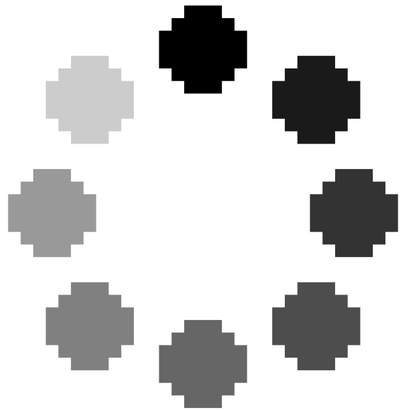
TIVIDY is a flexible engine built in C++ that brings workflow definitions to life as independent, real-time instances. It coordinates tasks, tracks state, and manages dynamic routing while keeping every process completely isolated.

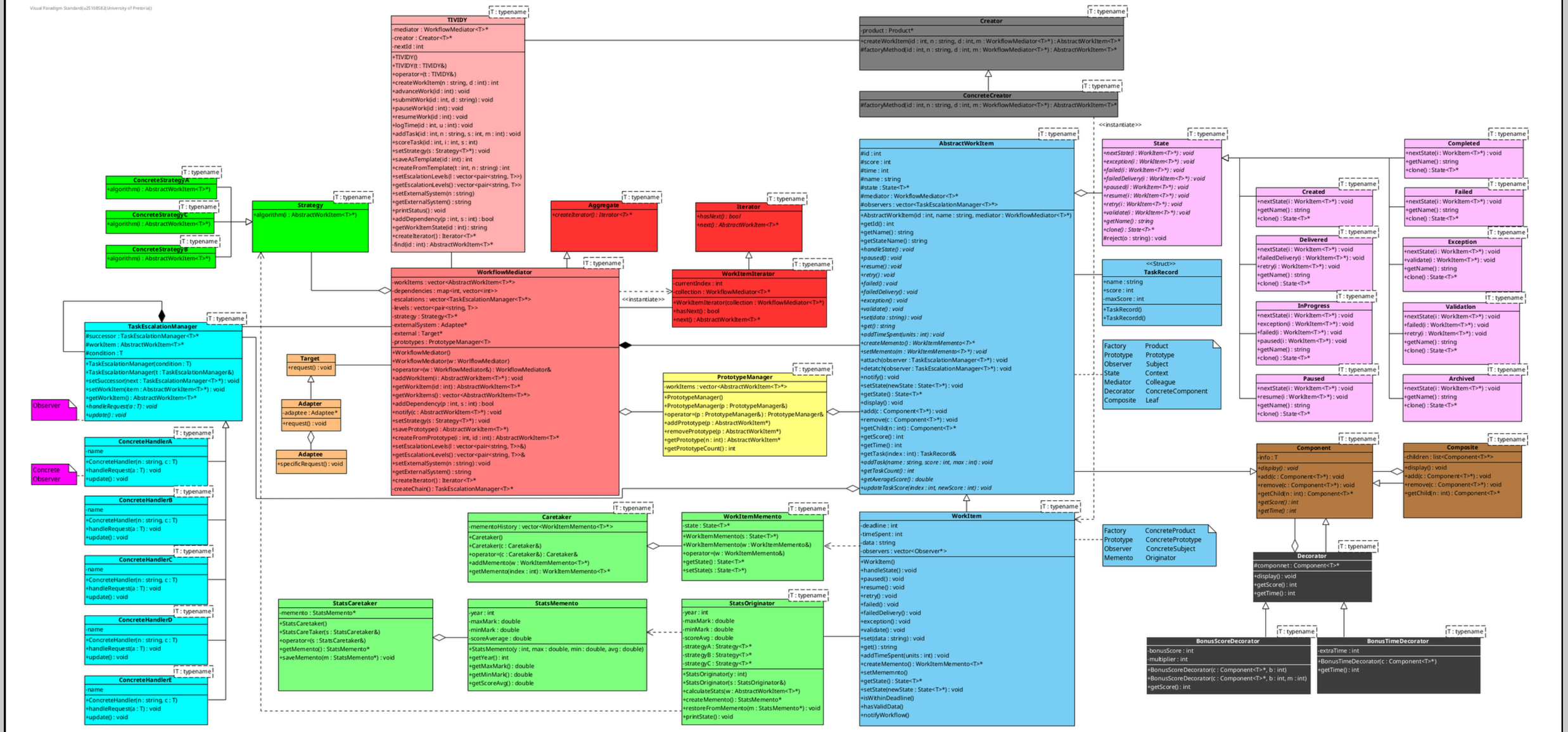
GoF Gonna Tell A Lie...

Home **Content** Live Demo Conclusion








Visual Paradigm Standard(u2510858(University of Pretoria))
TIVIDY
-mediator : WorkflowMediator<T>*
-creator : Creator<T>*
-nextId : int
+TIVIDY()
+TIVIDY(t : TIVIDY&)
+operator=(t : TIVIDY&)
+createWorkItem(n : string, d : int) : int
+advanceWork(id : int) : void
+submitWork(id : int, d : string) : void
+pauseWork(id : int) : void
+resumeWork(id : int) : void
+logTime(id : int, u : int) : void
+addTask(id : int, n : string, s : int, m : int) : void
+scoreTask(id : int, i : int, s : int)
+setStrategy(s : Strategy<T>*) : void
+saveAsTemplate(id : int) : int
+createFromTemplate(t : int, n : string) : int
+setEscalationLevels(l : vector<pair<string, T>>)
+getEscalationLevels() : vector<pair<string, T>>
+setExternalSystem(n : string)
+getExternalSystem() : string
+printStatus() : void
+addDependency(p : int, s : int) : bool
+getWorkItemState(id : int) : string
+createIterator() : Iterator<T>*
-find(id : int) : AbstractWorkItem<T>*
Creator
-product : Product*
+createWorkItem(id : int, n : string, d : int, m : WorkflowMediator<T>*) : AbstractWorkItem<T>*
#factoryMethod(id : int, n : string, d : int, m : WorkflowMediator<T>*) : AbstractWorkItem<T>*
ConcreteCreator
#factoryMethod(id : int, n : string, d : int, m : WorkflowMediator<T>*) : AbstractWorkItem<T>*
<<instantiate>>
ConcreteStrategyA
+algorithm() : AbstractWorkItem<T>*
ConcreteStrategyC
+algorithm() : AbstractWorkItem<T>*
ConcreteStrategyB
+algorithm() : AbstractWorkItem<T>*
Strategy
+algorithm() : AbstractWorkItem<T>*
Aggregate
+createIterator() : Iterator<T>*
Iterator
+hasNext() : bool
+next() : AbstractWorkItem<T>*
WorkflowMediator
-workItems : vector<AbstractWorkItem<T>*>
-dependencies : map<int, vector<int>>
-escalations : vector<TaskEscalationManager<T>*>
-levels : vector<pair<string, T>>
-strategy : Strategy<T>*
-externalSystem : Adaptee*
-external : Target*
-prototypes : PrototypeManager<T>
+WorkflowMediator()
+WorkflowMediator(w : WorlflowMediator)
+operator=(w : WorkflowMediator&) : WorkflowMediator&
+addWorkItem(i : AbstractWorkItem<T>*) : void
+getWorkItem(id : int) : AbstractWorkItem<T>*
+getWorkItems() : vector<AbstractWorkItem<T>*>
+addDependency(p : int, s : int) : bool
+notify(c : AbstractWorkItem<T>*) : void
+setStrategy(s : Strategy<T>*) : void
+savePrototype(i : AbstractWorkItem<T>*)
+createFromPrototype(i : int, id : int) : AbstractWorkItem<T>*
+setEscalationLevels(l : vector<pair<string, T>>&)
+getEscalationLevels() : vector<pair<string, T>>&
+setExternalSystem(n : string) : void
+getExternalSystem() : string
+createIterator() : Iterator<T>*
-createChain() : TaskEscalationManager<T>*
<<instantiate>>
WorkItemIterator
-currentIndex : int
-collection : WorkflowMediator<T>*
+WorkItemIterator(collection : WorkflowMediator<T>*)
+hasNext() : bool
+next() : AbstractWorkItem<T>*
AbstractWorkItem
#id : int
#score : int
#time : int
#name : string
#state : State<T>*
#mediator : WorkflowMediator<T>*
#observers : vector<TaskEscalationManager<T>*>
+AbstractWorkItem(id : int, name : string, mediator : WorkflowMediator<T>*)
+getId() : int
+getName() : string
+getStateName() : string
+handleState() : void
+paused() : void
+resume() : void
+retry() : void
+failed() : void
+failedDelivery() : void
+exception() : void
+validate() : void
+set(data : string) : void
+get() : string
+addTimeSpent(units : int) : void
+createMemento() : WorkItemMemento<T>*
+setMemento(m : WorkItemMemento<T>*) : void
+attach(observer : TaskEscalationManager<T>*) : void
+detatch(observer : TaskEscalationManager<T>*) : void
+notify() : void
+setState(newState : State<T>*) : void
+getState() : State<T>*
+display() : void
+add(c : Component<T>*) : void
+remove(c : Component<T>*) : void
+getChild(n : int) : Component<T>*
+getScore() : int
+getTime() : int
+getTask(index : int) : TaskRecord&
+addTask(name : string, score : int, max : int) : void
+getTaskCount() : int
+getAverageScore() : double
+updateTaskScore(index : int, newScore : int) : void
State
+nextState(i : WorkItem<T>*) : void
+exception() : WorkItem<T>*) : void
+failed() : WorkItem<T>*) : void
+failedDelivery() : WorkItem<T>*) : void
+paused() : WorkItem<T>*) : void
+resume() : WorkItem<T>*) : void
+retry() : WorkItem<T>*) : void
+validate() : WorkItem<T>*) : void
+getName() : string
+clone() : State<T>*
#reject(o : string) : void
<<Struct>>
TaskRecord
+name : string
+score : int
-maxScore : int
+TaskRecord()
+TaskRecordd()
Factory Product
Prototype Prototype
Observer Subject
State Context
Mediator Colleague
Decorator ConcreteComponent
Composite Leaf
Factory ConcreteProduct
Prototype ConcretePrototype
Observer ConcreteSubject
Memento Originator
Completed
+nextState(i : WorkItem<T>*) : void
+getName() : string
+clone() : State<T>*
Created
+nextState(i : WorkItem<T>*) : void
+getName() : string
+clone() : State<T>*
Failed
+nextState(i : WorkItem<T>*) : void
+getName() : string
+clone() : State<T>*
Delivered
+nextState(i : WorkItem<T>*) : void
+failedDelivery(i : WorkItem<T>*) : void
+retry(i : WorkItem<T>*) : void
+getName() : string
+clone() : State<T>*
Exception
+nextState(i : WorkItem<T>*) : void
+validate() : WorkItem<T>*) : void
+getName() : string
+clone() : State<T>*
InProgress
+nextState(i : WorkItem<T>*) : void
+exception(i : WorkItem<T>*) : void
+failed(i : WorkItem<T>*) : void
+paused(i : WorkItem<T>*) : void
+getName() : string
+clone() : State<T>*
Validation
+nextState(i : WorkItem<T>*) : void
+failed(i : WorkItem<T>*) : void
+retry(i : WorkItem<T>*) : void
+getName() : string
+clone() : State<T>*
Paused
+nextState(i : WorkItem<T>*) : void
+resume(i : WorkItem<T>*) : void
+getName() : string
+clone() : State<T>*
Archived
+nextState(i : WorkItem<T>*) : void
+getName() : string
+clone() : State<T>*
TaskEscalationManager
#successor : TaskEscalationManager<T>*
#workItem : AbstractWorkItem<T>*
#condition : T
+TaskEscalationManager(condition : T)
+TaskEscalationManager(t : TaskEscalationManager&)
+setSuccessor(next : TaskEscalationManager<T>*) : void
+setWorkItem(item : AbstractWorkItem<T>*) : void
+getWorkItem() : AbstractWorkItem<T>*
+handleRequest(a : T) : void
+update() : void
Observer
Concrete Observer
ConcreteHandlerA
-name
+ConcreteHandler(n : string, c : T)
+handleRequest(a : T) : void
+update() : void
ConcreteHandlerB
-name
+ConcreteHandler(n : string, c : T)
+handleRequest(a : T) : void
+update() : void
ConcreteHandlerC
-name
+ConcreteHandler(n : string, c : T)
+handleRequest(a : T) : void
+update() : void
ConcreteHandlerD
-name
+ConcreteHandler(n : string, c : T)
+handleRequest(a : T) : void
+update() : void
ConcreteHandlerE
-name
+ConcreteHandler(n : string, c : T)
+handleRequest(a : T) : void
+update() : void
Target
+request() : void
Adapter
-adaptee : Adaptee*
+request() : void
Adaptee
+specificRequest() : void
PrototypeManager
-workItems : vector<AbstractWorkItem<T>*>
+PrototypeManager()
+PrototypeManager(p : PrototypeManager&)
+operator=(p : PrototypeManager&) : PrototypeManager&
+addPrototype(p : AbstractWorkItem*)
+removePrototype(p : AbstractWorkItem*)
+getPrototype(n : int) : AbstractWorkItem*
+getPrototypeCount() : int
Caretaker
-mementoHistory : vector<WorkItemMemento<T>*>
+Caretaker()
+Caretaker(c : Caretaker&)
+operator=(c : Caretaker&) : Caretaker&
+addMemento(w : WorkItemMemento<T>*)
+getMemento(index : int) : WorkItemMemento<T>*
WorkItemMemento
-state : State<T>*
+WorkItemMemento(s : State<T>*)
+WorkItemMemento(w : WorkItemMemento&)
+operator=(w : WorkItemMemento&)
+getState() : State<T>*
+setState(s : State<T>*)
WorkItem
-deadline : int
-timeSpent : int
-data : string
-observers : vector<Observer*>
+WorkItem()
+handleState() : void
+paused() : void
+resume() : void
+retry() : void
+failed() : void
+failedDelivery() : void
+exception() : void
+validate() : void
+set(data : string) : void
+get() : string
+addTimeSpent(units : int) : void
+createMemento() : WorkItemMemento<T>*
+setMememto()
+getState() : State<T>*
+setState(newState : State<T>*) : void
+isWithinDeadline()
+hasValidData()
+notifyWorkflow()
Component
-info : T
+display() : void
+add(c : Component<T>*) : void
+remove(c : Component<T>*) : void
+getChild(n : int) : Component<T>*
+getScore() : int
+getTime() : int
Composite
-children : list<Component<T>*>
+display() : void
+add(c : Component<T>*) : void
+remove(c : Component<T>*) : void
+getChild(n : int) : Component<T>*
Decorator
#componnet : Component<T>*
+display() : void
+getScore() : int
+getTime() : int
BonusScoreDecorator
-bonusScore : int
-multiplier : int
+BonusScoreDecorator(c : Component<T>*, b : int)
+BonusScoreDecorator(c : Component<T>*, b : int, m : int)
+getScore() : int
BonusTimeDecorator
-extraTime : int
+BonusTimeDecorator(c : Component<T>*)
+getTime() : int
StatsCaretaker
-memento : StatsMemento*
+StatsCaretaker()
+StatsCareTaker(s : StatsCaretaker&)
+operator=(s : StatsCaretaker&)
+getMemento() : StatsMemento*
+saveMemento(m : StatsMemento*) : void
StatsMemento
-year : int
-maxMark : double
-minMark : double
-scoreAverage : double
+StatsMemento(y : int, max : double, min : double, avg : double)
+getYear() : int
+getMaxMark() : double
+getMinMark() : double
+getScoreAvg() : double
StatsOriginator
-year : int
-maxMark : double
-minMark : double
-scoreAvg : double
-strategyA : Strategy<T>*
-strategyB : Strategy<T>*
-strategyC : Strategy<T>*
+StatsOriginator(y : int)
+StatsOriginator(s : StatsOriginator&)
+calculateStats(w : AbstractWorkItem<T>*)
+createMemento() : StatsMemento*
+restoreFromMemento(m : StatsMemento*) : void
+printState() : void

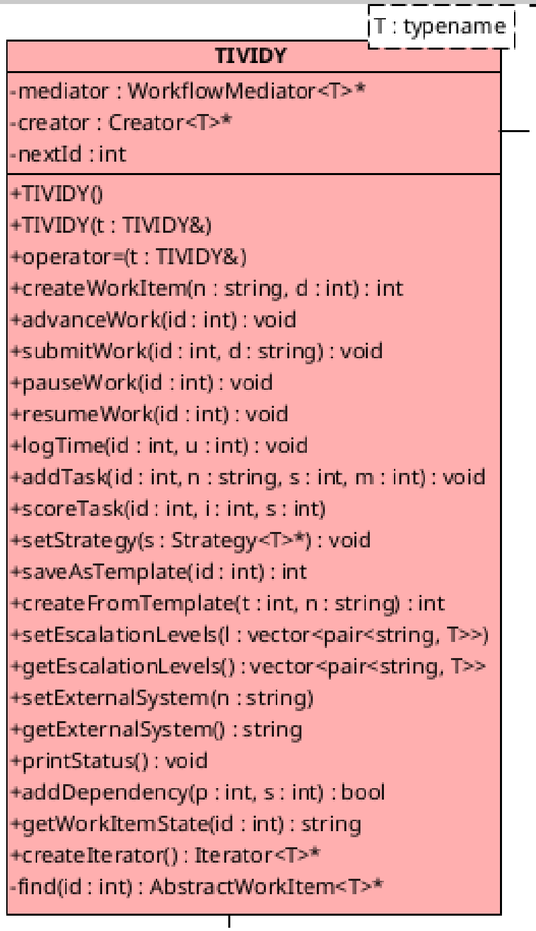
PATTERNS

- Facade
- Factory Method
- Mediator
- Strategy
- Prototype
- Adapter
- Chain of Responsibility
- Observer
- Composite
- Decorator
- State
- Memento



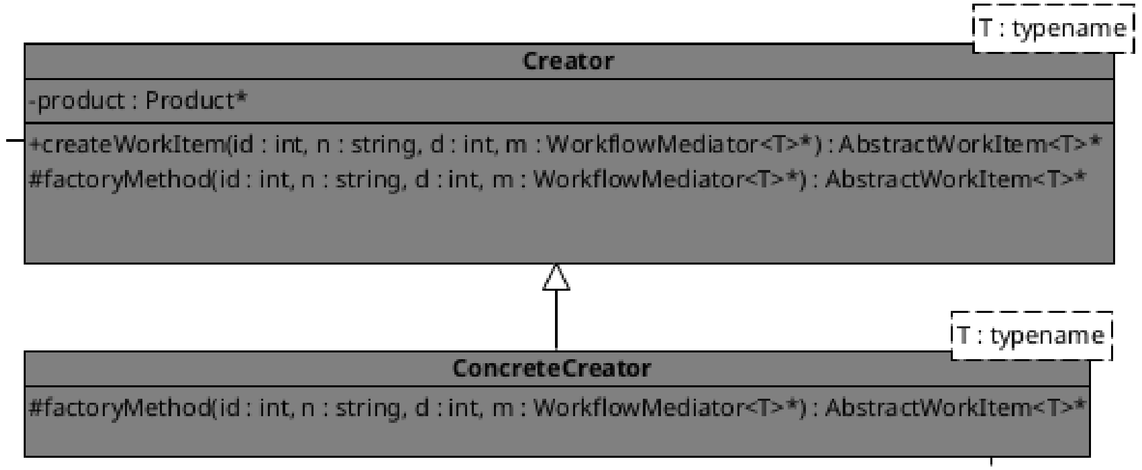
FACADE

The Facade pattern is used to give our system a single, simple front door through the TIVIDY class, hiding all the underlying complexity.



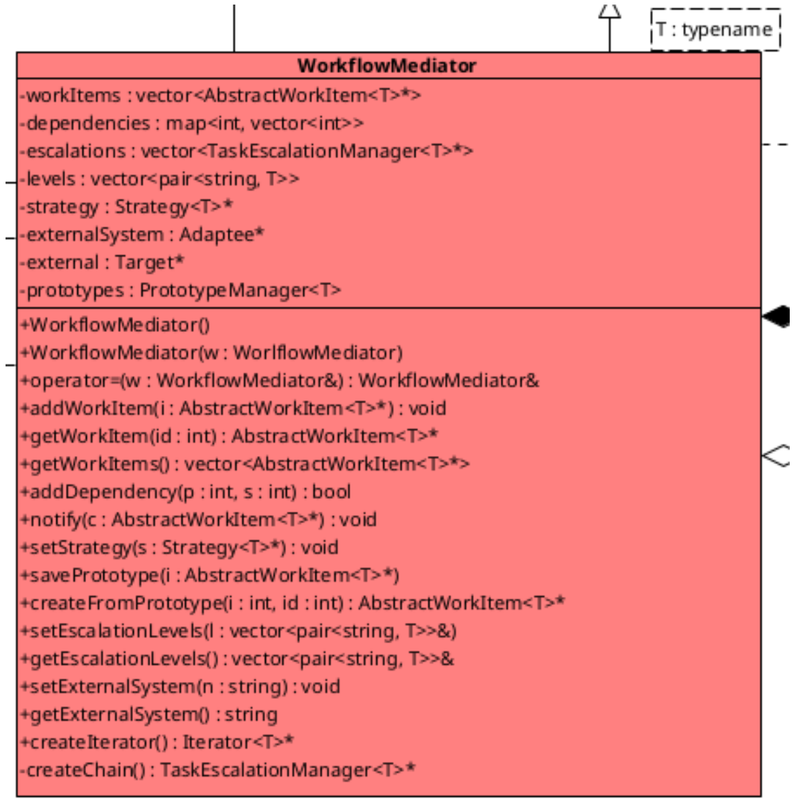
FACTORY

Factory Method handles creating WorkItem behind the scenes, keeping the rest of the system decoupled from specific object instantiation.



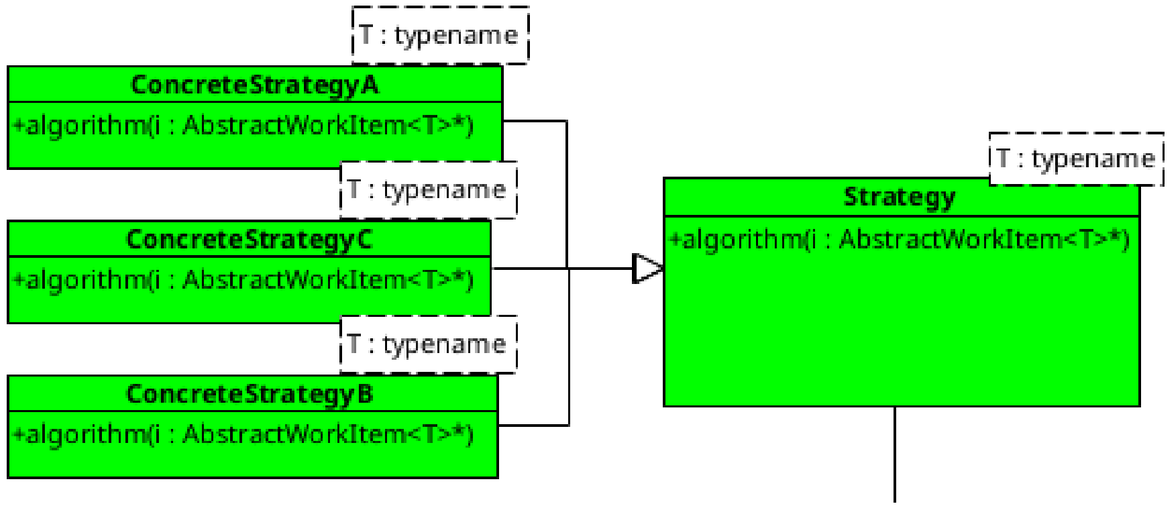
MEDIATOR

The Mediator acts as a traffic controller, letting work items, adapters, and the main system talk to each other without tying everything into a tangled mess.



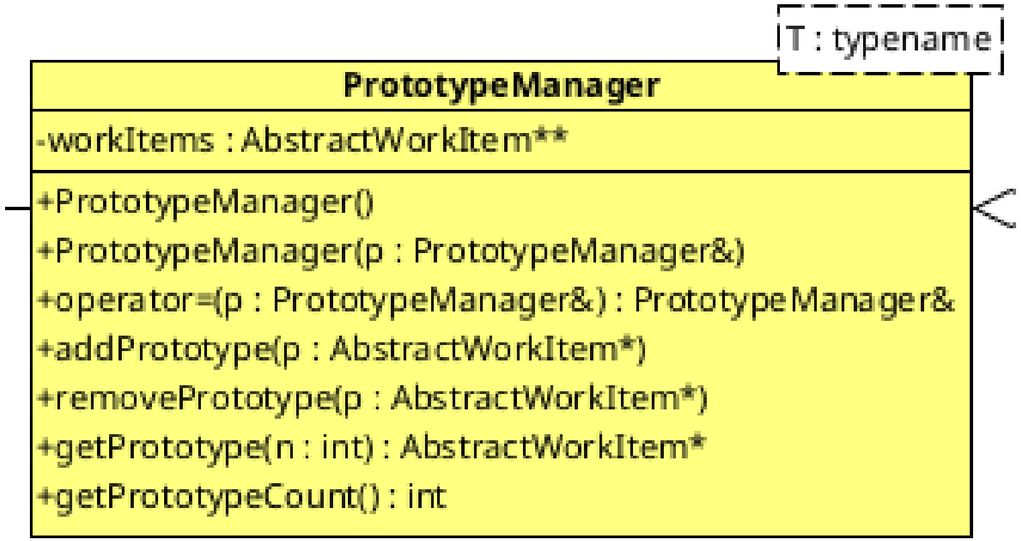
STRATEGY

The Strategy pattern lets us swap out statistical algorithms on the fly whenever we need to calculate different stats for a work item.



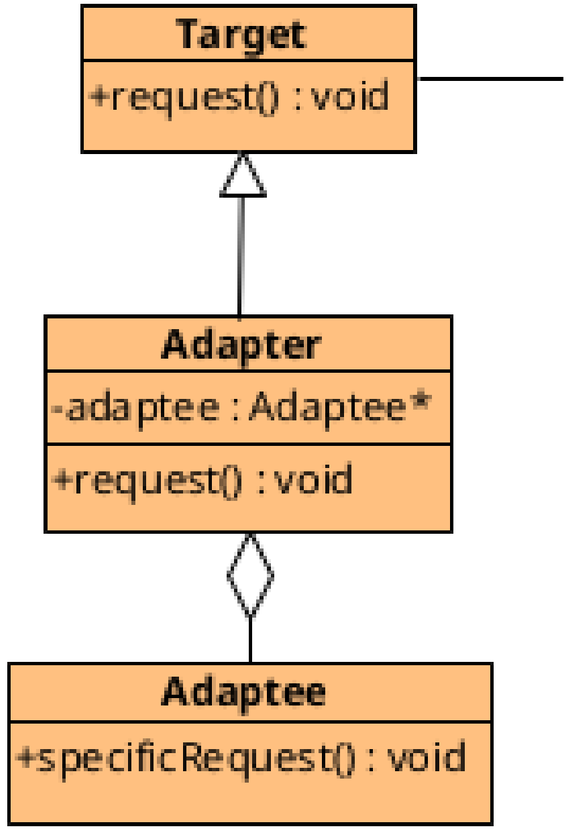
PROTOTYPE

With the Prototype pattern, our PrototypeManager can clone existing WorkItem whenever we need fresh copies without having to rebuild them from scratch.



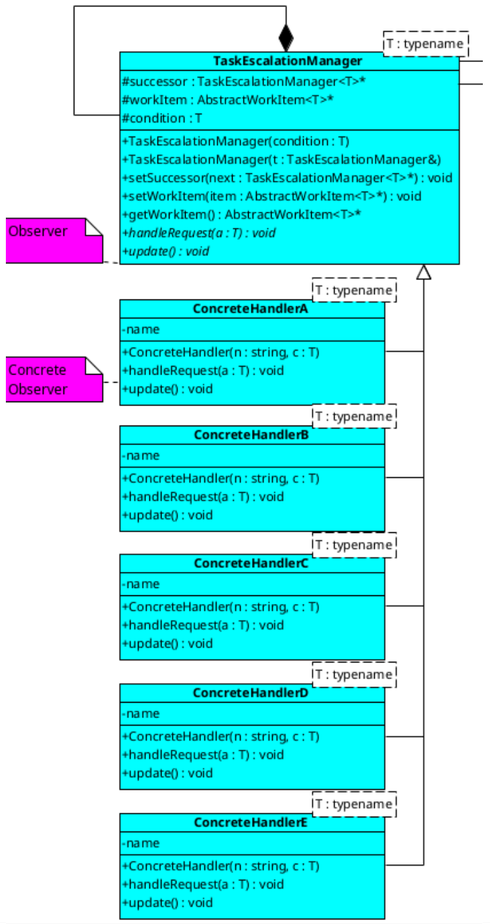
ADAPTER

The Adapter pattern acts as a bridge, taking an external system and translating it into a format our system understands.



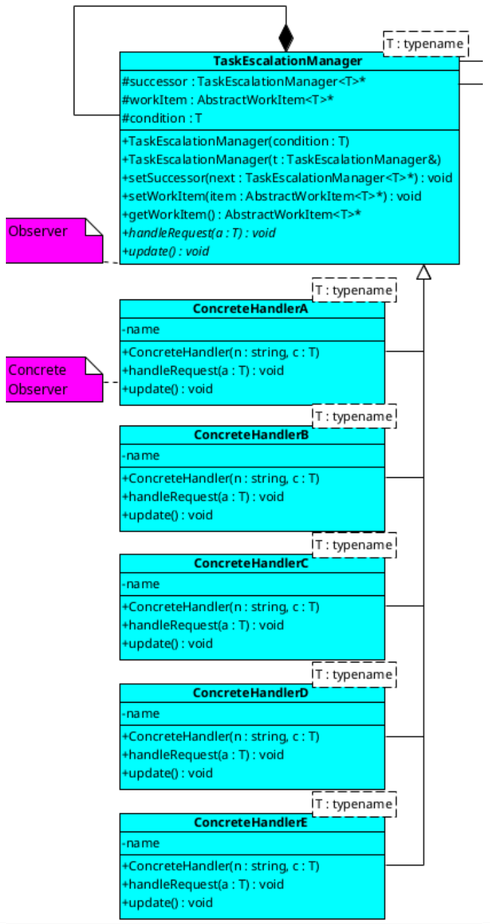
CHAIN

The Chain of Responsibility passes a WorkItem down a line of handlers when something changes, letting the right handler step up and process it.



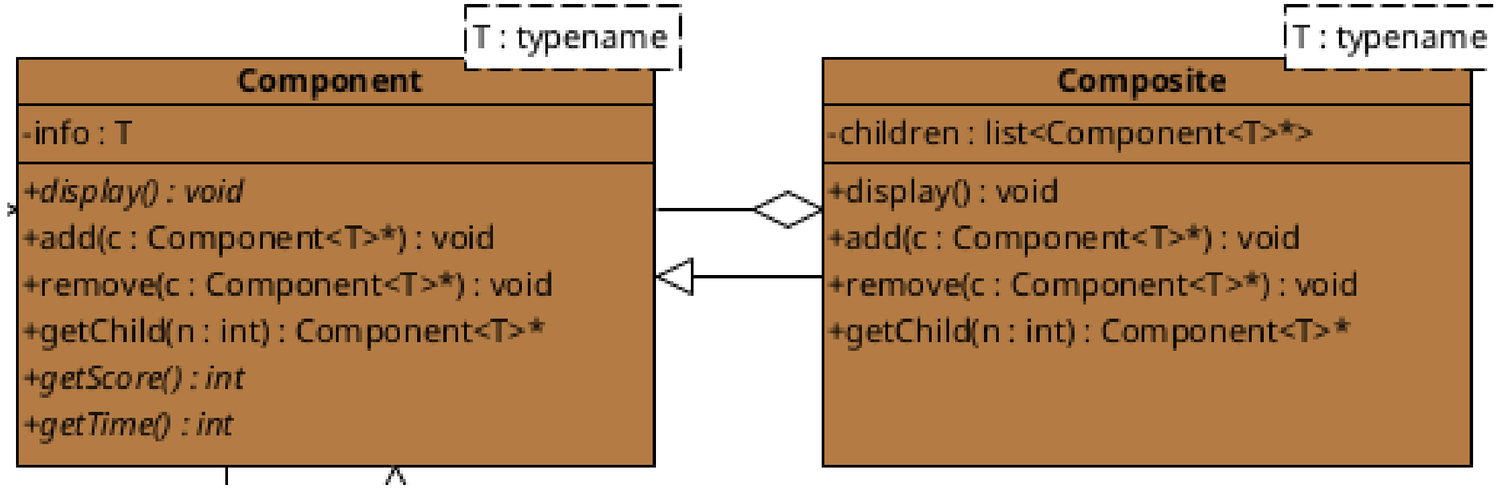
OBSERVER

The Observer pattern keeps an eye on WorkItem and automatically alerts our Chain of Responsibility the moment a state change happens.



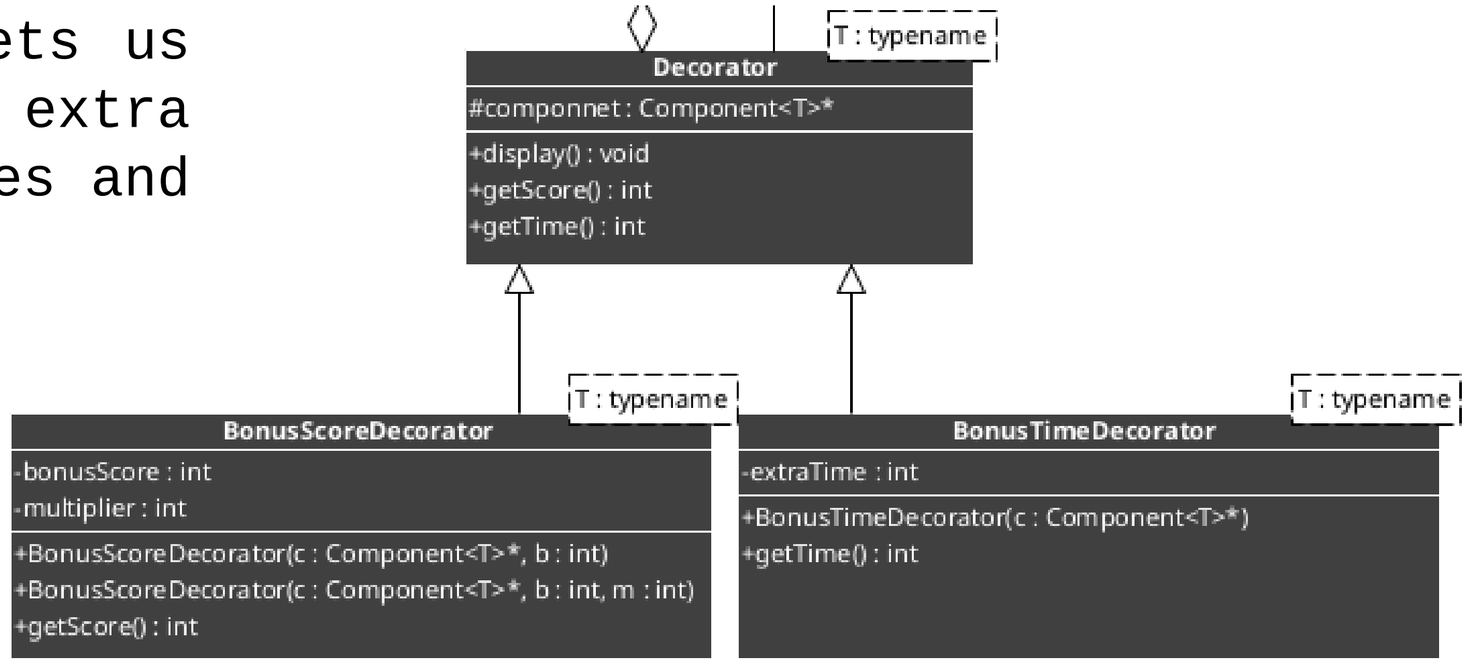
COMPOSITE

The Composite pattern organizes our components into a WorkItem hierarchy.



DECORATOR

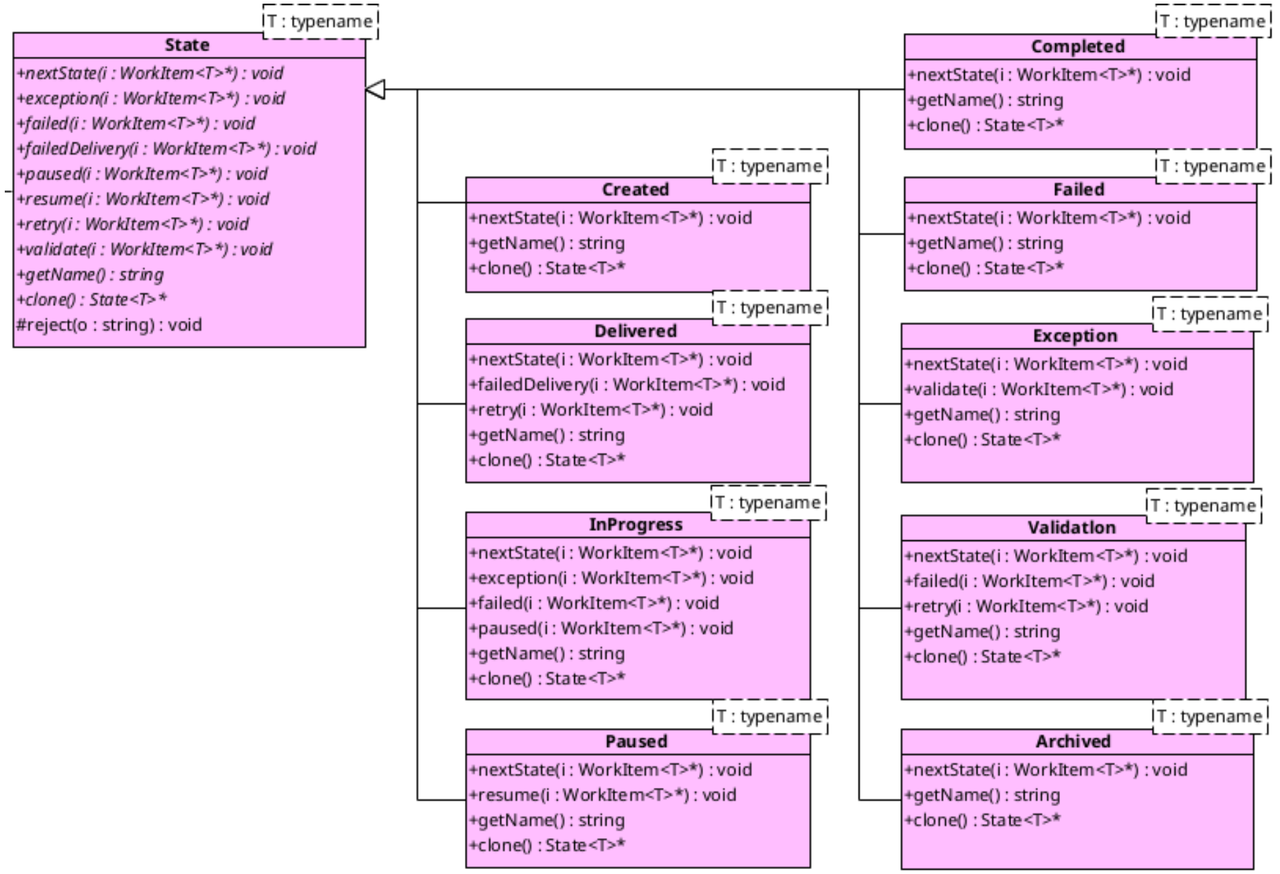
The Decorator pattern lets us wrap the WorkItem with extra features like bonus scores and additional time.



Ooh (GoF Gonna Give, GoF Gonna Give)...

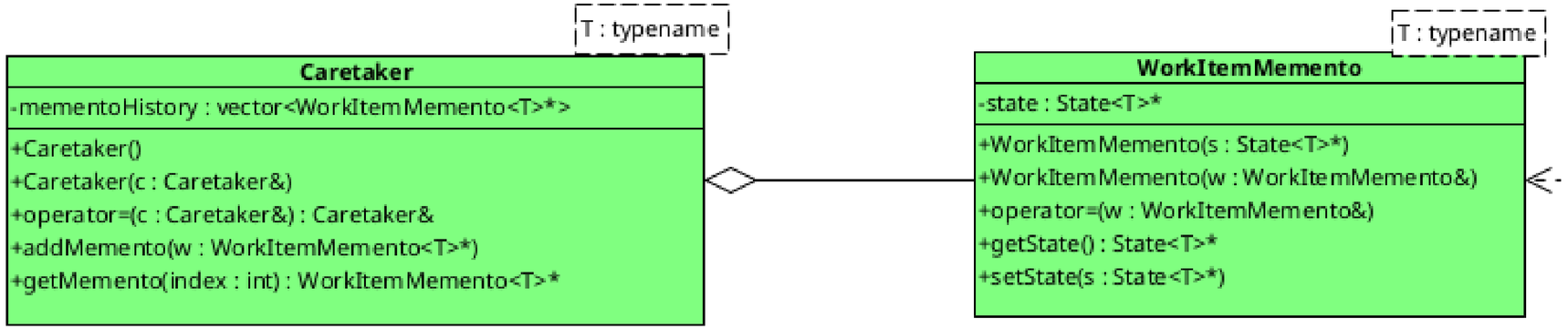
STATE

The State pattern allows to control the lifecycle of the WorkItem.



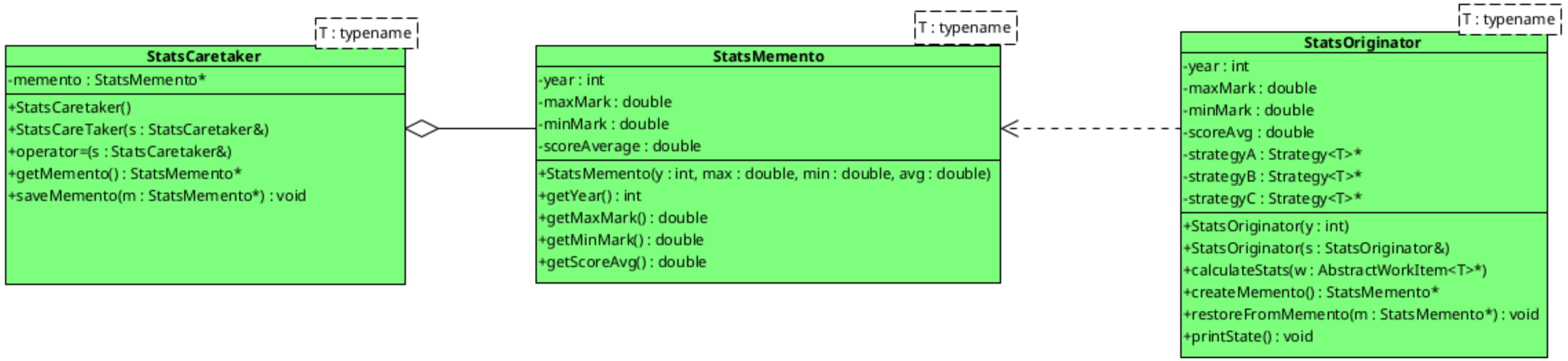
MEMENTO.1

We used the Memento pattern with WorkItemMemento to take snapshots of a WorkItem's current state, allowing us to safely restore its progress whenever needed.



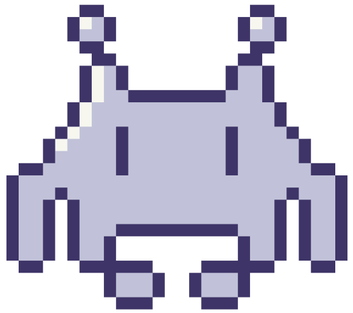
MEMENTO.2

Memento.2 uses the Strategy pattern to caluclate statistics from the WorkItem and saves a snapshot of the statistics.



Ooh-ooh (Never gonna give, never gonna give)...

Home **Content** Live Demo Conclusion



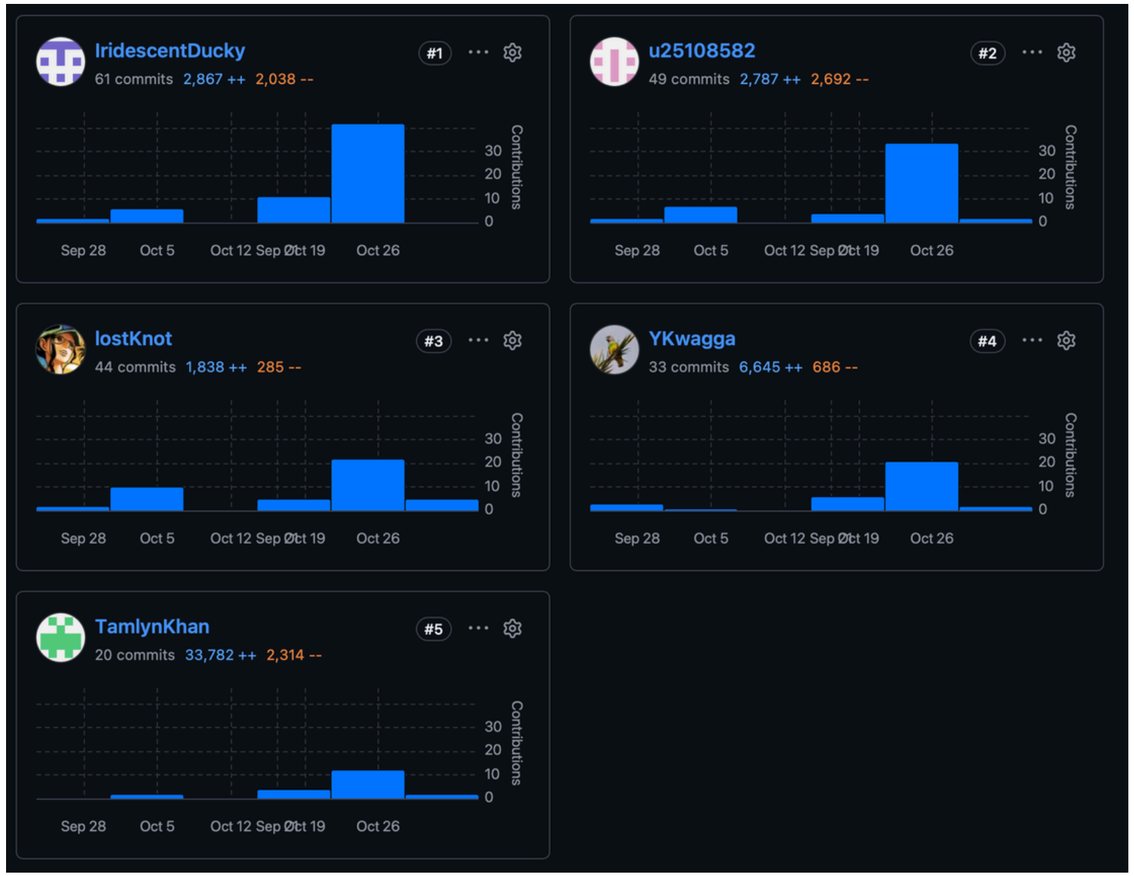
GITHUB

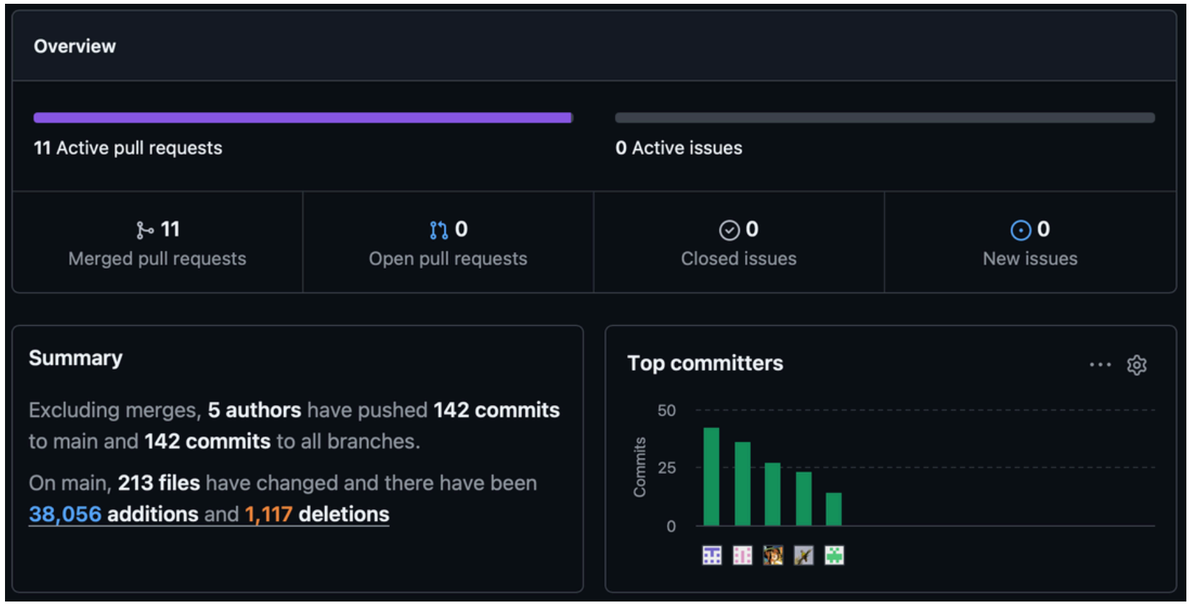
Commit

Pull

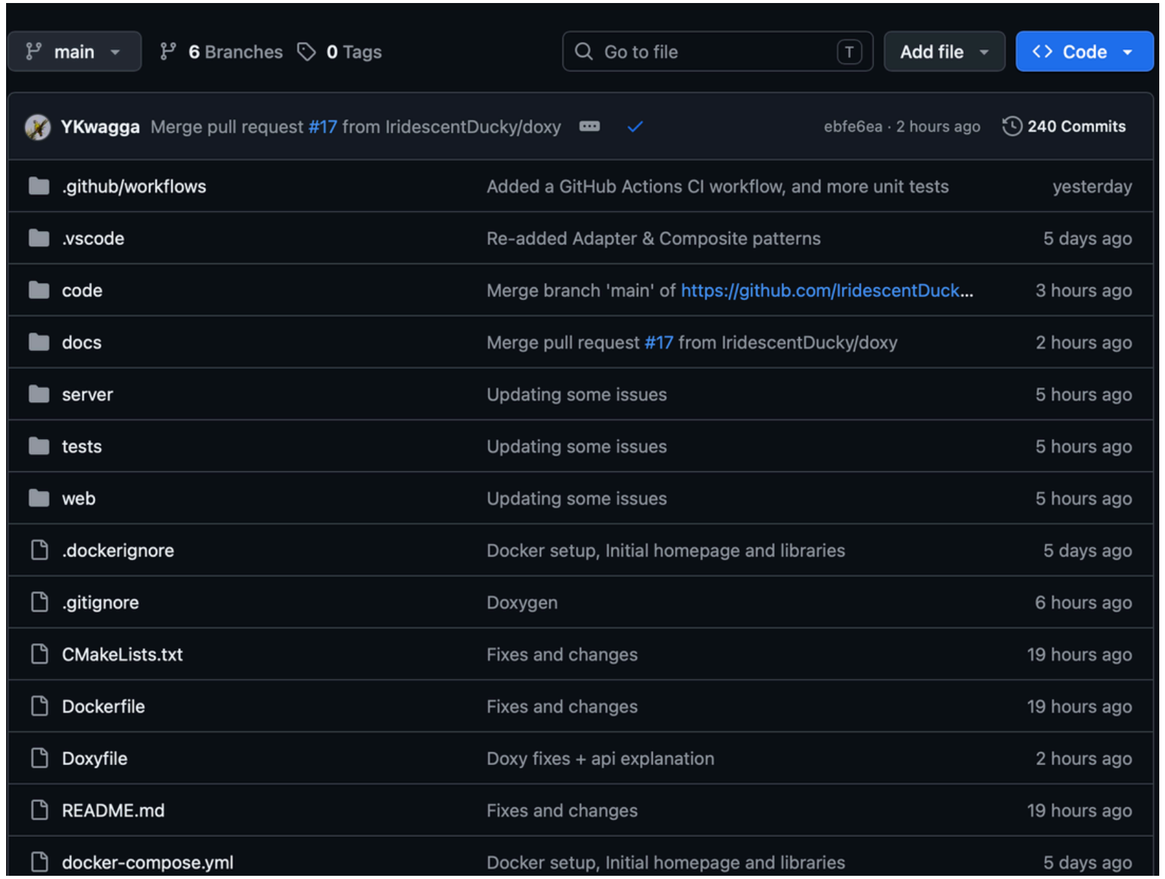
Give you up...

Home **Content** Live Demo Conclusion



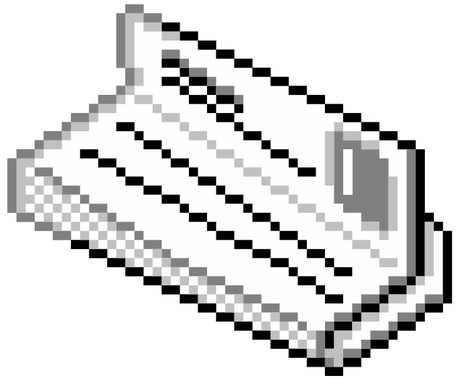


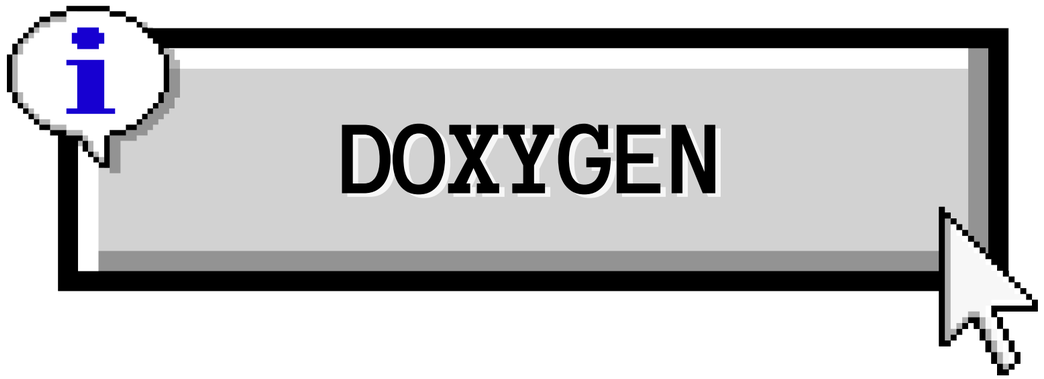
We've known each other for so long...

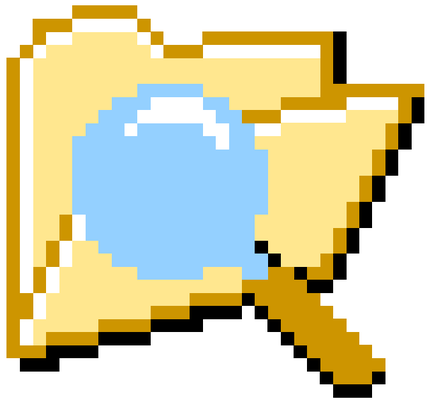


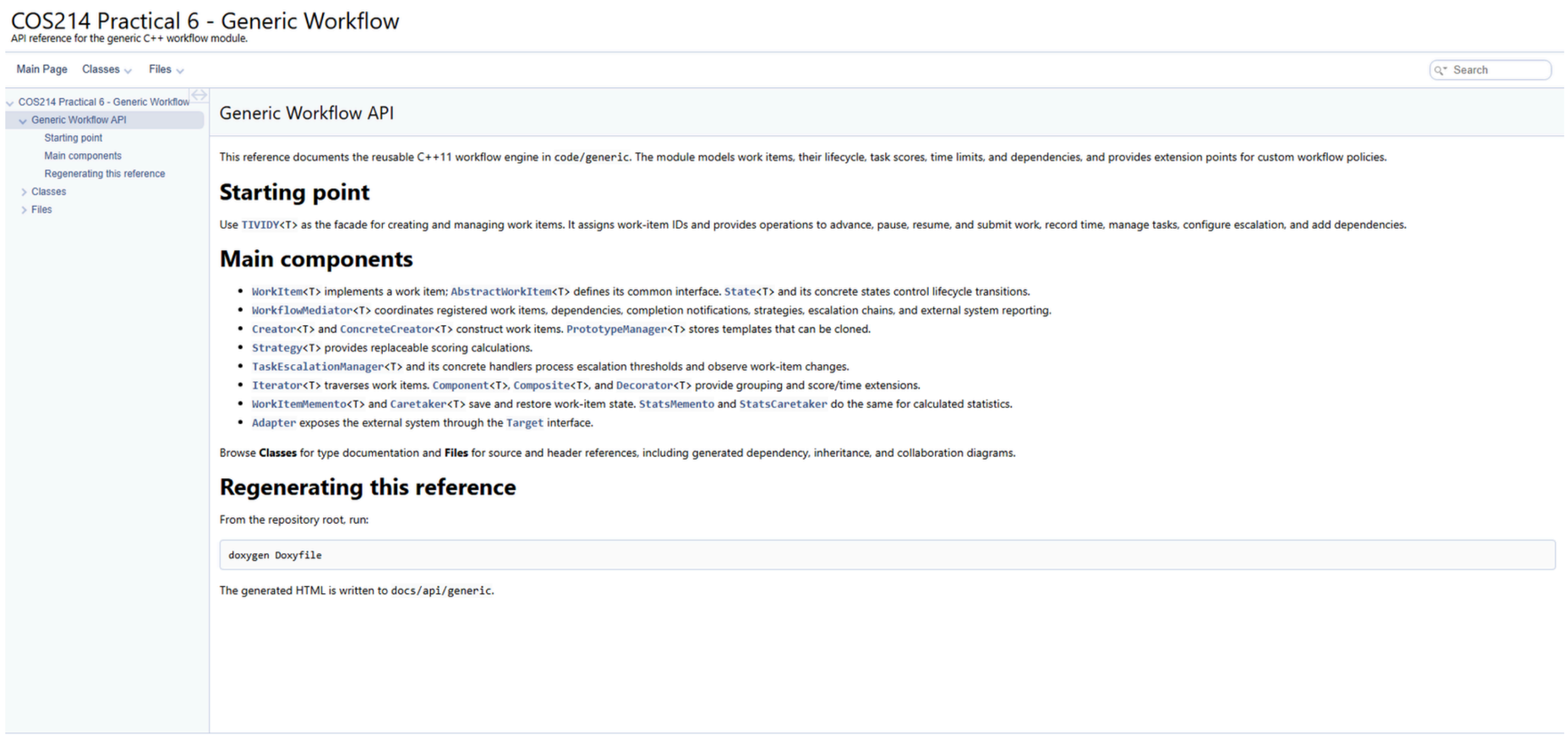
Your heart's been aching...

Home **Content** Live Demo Conclusion








COS214 Practical 6 - Generic Workflow
API reference for the generic C++ workflow module.
Main Page
Classes
Files
Search
COS214 Practical 6 - Generic Workflow
Generic Workflow API
Starting point
Main components
Regenerating this reference
Classes
Files
Generic Workflow API
This reference documents the reusable C++11 workflow engine in code/generic. The module models work items, their lifecycle, task scores, time limits, and dependencies, and provides extension points for custom workflow policies.
Starting point
Use TIVIDY<T> as the facade for creating and managing work items. It assigns work-item IDs and provides operations to advance, pause, resume, and submit work, record time, manage tasks, configure escalation, and add dependencies.
Main components
WorkItem<T> implements a work item; AbstractWorkItem<T> defines its common interface. State<T> and its concrete states control lifecycle transitions.
WorkflowMediator<T> coordinates registered work items, dependencies, completion notifications, strategies, escalation chains, and external system reporting.
Creator<T> and ConcreteCreator<T> construct work items. PrototypeManager<T> stores templates that can be cloned.
Strategy<T> provides replaceable scoring calculations.
TaskEscalationManager<T> and its concrete handlers process escalation thresholds and observe work-item changes.
Iterator<T> traverses work items. Component<T>, Composite<T>, and Decorator<T> provide grouping and score/time extensions.
WorkItemMemento<T> and Caretaker<T> save and restore work-item state. StatsMemento and StatsCaretaker do the same for calculated statistics.
Adapter exposes the external system through the Target interface.
Browse Classes for type documentation and Files for source and header references, including generated dependency, inheritance, and collaboration diagrams.
Regenerating this reference
From the repository root, run:
doxygen Doxyfile
The generated HTML is written to docs/api/generic.

Inside, we both know what's been going on...


```cpp
/** @brief Score and maximum score for a named task within a work item. */
struct TaskRecord {
    std::string name;
    int score;
    int maxScore;
    bool scored;

    /** @brief Creates an empty task record. */
    TaskRecord() : name(""), score(0), maxScore(0), scored(false) {}
    /** @brief Creates a task record with the supplied score values.
     *  @param n Task name.
     *  @param s Current task score.
     *  @param ms Maximum task score.
     */
    TaskRecord(const std::string& n, int s, int ms) : name(n), score(s), maxScore(ms), scored(tr
    /** @brief Creates a task record that has not been scored yet.
     *  @param n Task name.
     *  @param ms Maximum task score.
     */
    TaskRecord(const std::string& n, int ms) : name(n), score(0), maxScore(ms), score(false) {}
};
```

```cpp
/** @brief Base decorator that forwards component operations. */
template <typename T>
class Decorator : public Component<T> {
public:

    /** @brief Wraps a component with decorator behavior.
     *  @param component Component being decorated.
     */
    Decorator(Component<T>* component);
    /** @brief Displays the wrapped component.
     *  @param indent Indentation level used for hierarchical output.
     */
    void display(int indent = 0);
    /** @brief Destroys the decorator. */
    virtual ~Decorator();

    /** @brief Gets the wrapped component's score. */
    int getScore();
    /** @brief Gets the wrapped component's time allowance. */
    int getTime();

protected:
    Component<T>* component ;//component to deco
};
```

Constructs a concrete work item.

Parameters

id	Work-item identifier.
name	Work-item name.
deadline	Allowed work duration.
mediator	Mediator coordinating the work item.

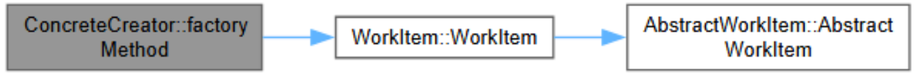
Returns

The newly created work item.

Implements **Creator< T >**.

Definition at line **8** of file **ConcreteCreator.cpp**.

Here is the call graph for this function:



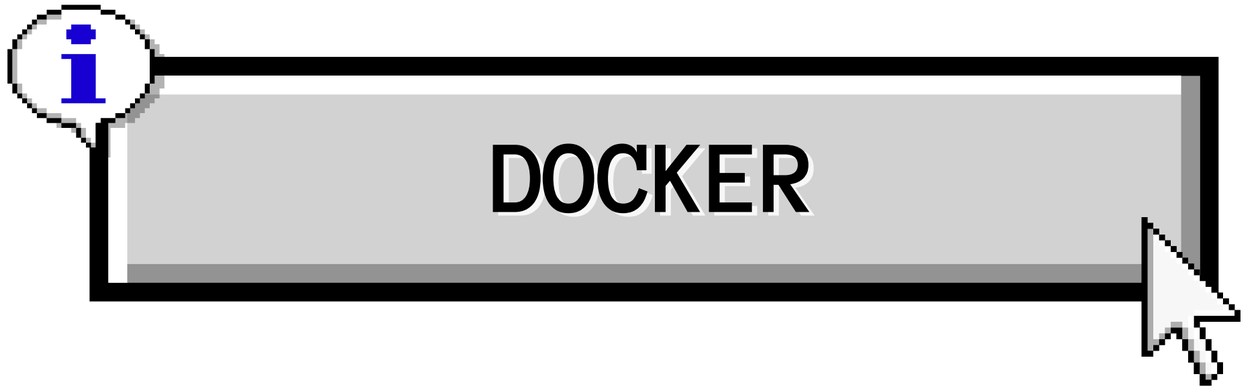
The documentation for this class was generated from the following files:

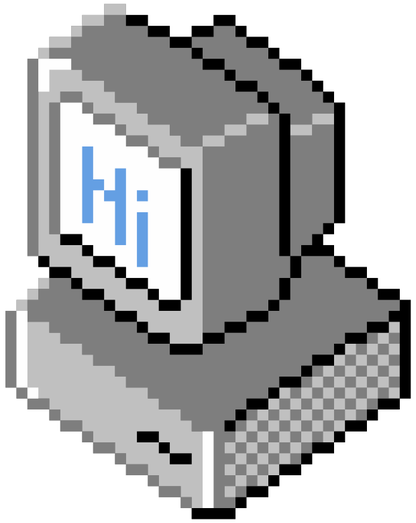
- code/generic/**ConcreteCreator.h**
- code/generic/**ConcreteCreator.cpp**

and we're gonna play it...

Home **Content** Live Demo Conclusion

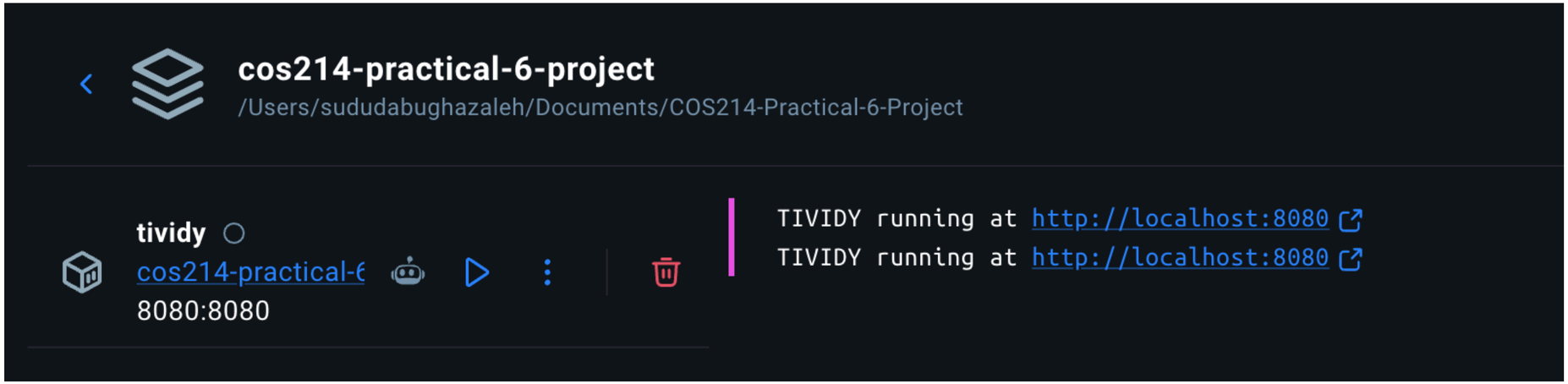






I just wanna tell you how I'm feeling...

Home **Content** Live Demo Conclusion



Gotta make you understand...

Home **Content** Live Demo Conclusion

```
tamly@BumbleBee:~/cos214/COS214_PA6/COS214-Practical-6-Project$ docker compose up --build
WARN[0001] Docker Compose is configured to build using Bake, but buildx isn't installed
[+] Building 1.3s (16/16) FINISHED                                                                                  docker:default
 => [tividy internal] load build definition from Dockerfile                                                                 0.1s
 => => transferring dockerfile: 433B                                                                                        0.1s
 => [tividy internal] load metadata for docker.io/library/debian:bookworm-slim                                              0.1s
 => [tividy internal] load metadata for docker.io/library/gcc:13.2.0                                                        0.1s
 => [tividy internal] load .dockerignore                                                                                    0.0s
 => => transferring context: 50B                                                                                            0.0s
 => [tividy internal] load build context                                                                                    0.1s
 => => transferring context: 3.63kB                                                                                         0.1s
 => [tividy build 1/5] FROM docker.io/library/gcc:13.2.0@sha256:15c73bc59ae88b3fd563ef2ec4a8743a8848a9f74362b6d116c4543c4844b6e0  0.1s
 => => resolve docker.io/library/gcc:13.2.0@sha256:15c73bc59ae88b3fd563ef2ec4a8743a8848a9f74362b6d116c4543c4844b6e0          0.1s
 => [tividy stage-1 1/4] FROM docker.io/library/debian:bookworm-slim@sha256:99c0b25c46d0214d5a2091092e7908907dcaecd6a962cde0ac700acee222b6f3  0.1s
 => => resolve docker.io/library/debian:bookworm-slim@sha256:99c0b25c46d0214d5a2091092e7908907dcaecd6a962cde0ac700acee222b6f3  0.1s
 => CACHED [tividy stage-1 2/4] WORKDIR /app                                                                                0.0s
 => CACHED [tividy build 2/5] WORKDIR /src                                                                                  0.0s
 => CACHED [tividy build 3/5] COPY server/ server/                                                                          0.0s
 => CACHED [tividy build 4/5] COPY code/generic/ code/generic/                                                              0.0s
 => CACHED [tividy build 5/5] RUN g++ -std=c++11 -O2 -pthread -static-libstdc++ -static-libgcc -Icode/generic -Iserver     server/*.cpp $(find code/gene  0.0s
 => CACHED [tividy stage-1 3/4] COPY --from=build /src/tividy .                                                             0.0s
 => CACHED [tividy stage-1 4/4] COPY web/generic/ web/generic/                                                              0.0s
 => [tividy] exporting to image                                                                                             0.4s
 => => exporting layers                                                                                                     0.0s
 => => exporting manifest sha256:0b7dbd519f092bdb809d0aff96dd67b96f1cd3a5d19765dfa4e300f2b6695a1b                           0.0s
 => => exporting config sha256:e32c530ced1137d79b1f77172346a079a3f3be84c544e6204052f14802e86221                             0.0s
 => => exporting attestation manifest sha256:00100b8de4088f91ac5afb1e61e1c2bcd4f3fd7f0794a2bec5b73a4bfadb0291               0.1s
 => => exporting manifest list sha256:3a638422bb9fecec86f01421fe949dce51748027b34fb6de721d25fcbc935ce0                      0.0s
 => => naming to docker.io/library/cos214-practical-6-project-tividy:latest                                                 0.0s
 => => unpacking to docker.io/library/cos214-practical-6-project-tividy:latest                                              0.0s
 => [tividy] resolving provenance for metadata file                                                                         0.0s
[+] Running 2/2
 ✓ tividy                                  Built                                                                            0.0s
 ✓ Container cos214-practical-6-project-tividy-1  Recreated                                                                11.6s
```

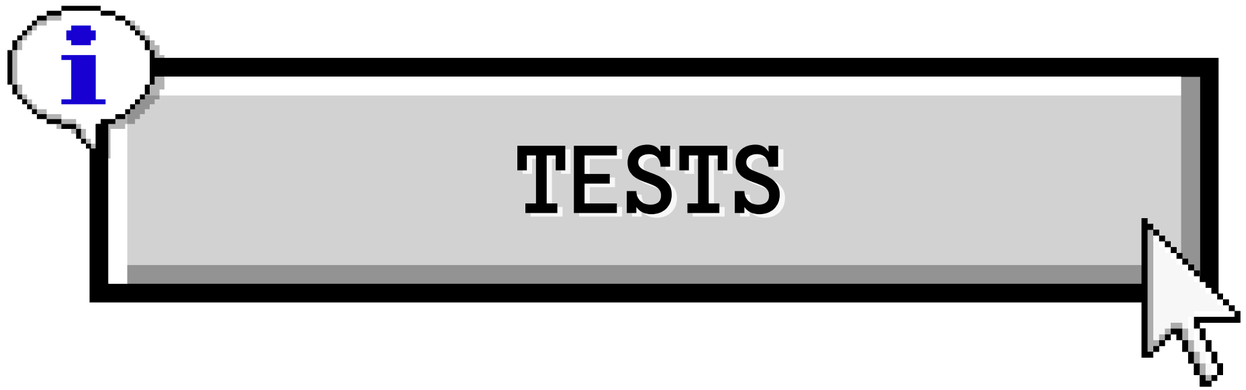
GOF Gonna Give You Up...

Home **Content** Live Demo Conclusion

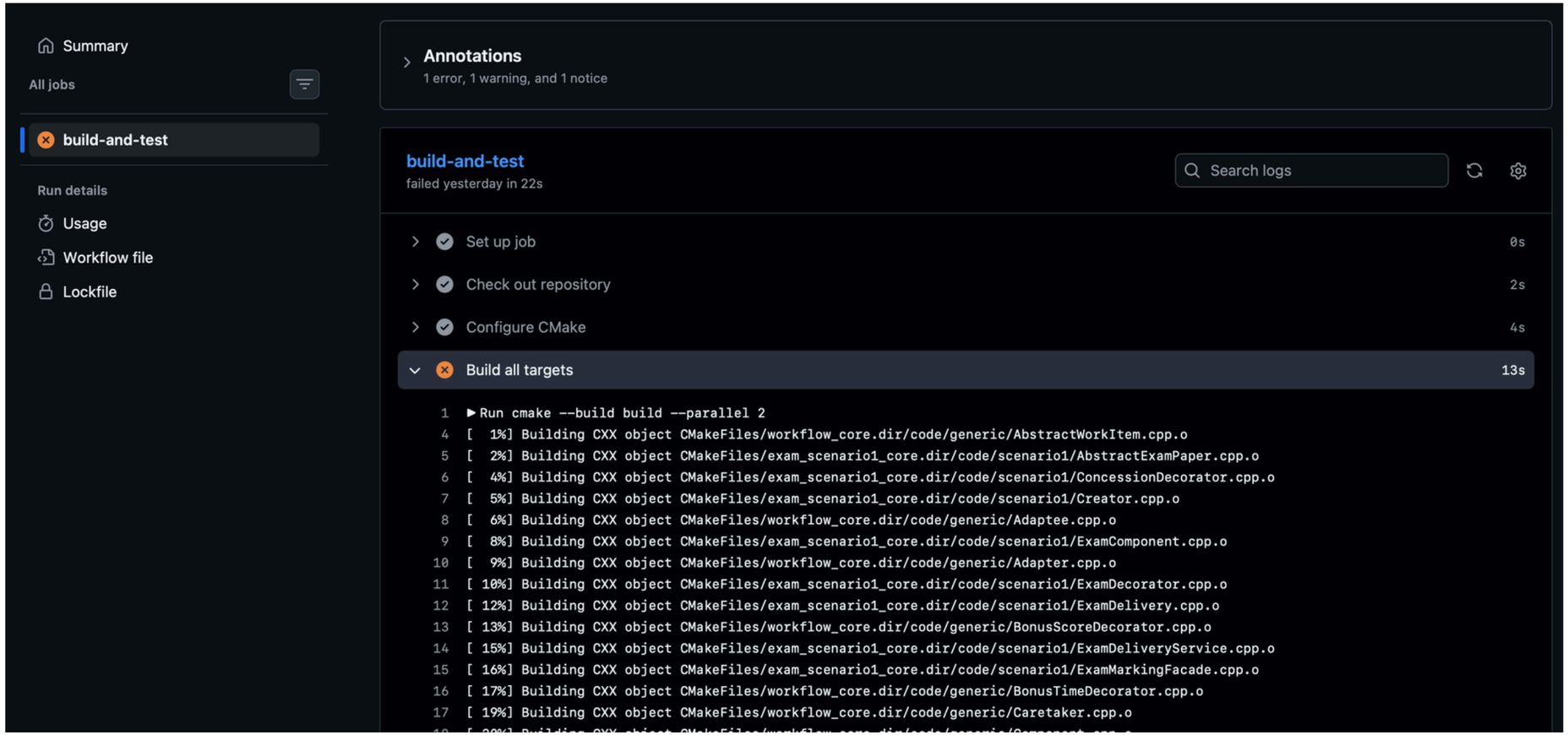
```
[+] Running 2/2
 ✓ tividy                                      Built
 ✓ Container cos214-practical-6-project-tividy-1  Recreated
Attaching to tividy-1
tividy-1  | TIVIDY running at http://localhost:8080
```

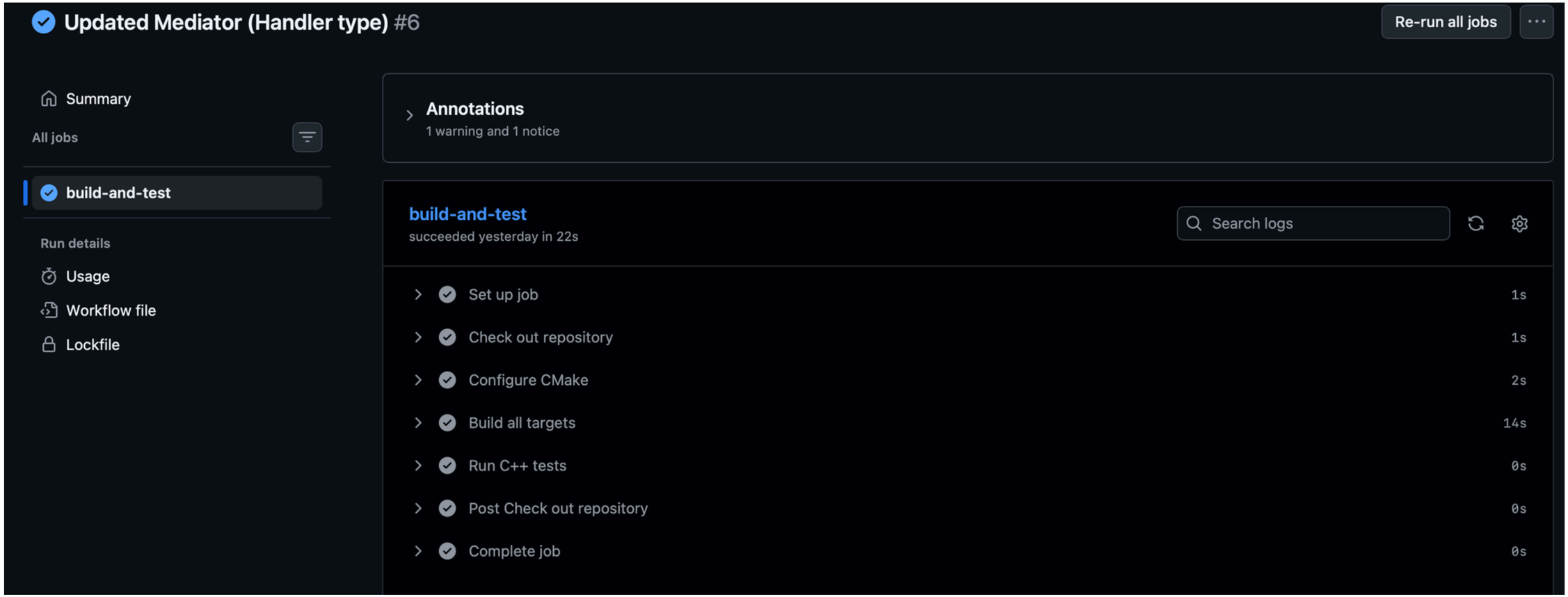
GOF Gonna Let You Down...

Home **Content** Live Demo Conclusion



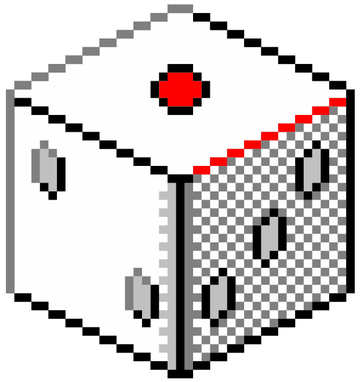
GOF Gonna Run Around...



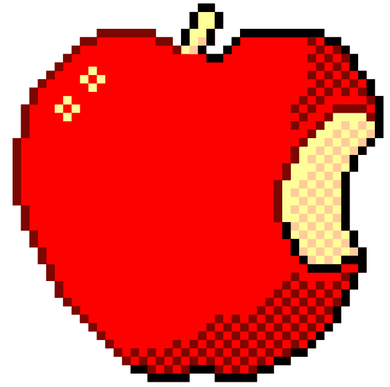

Updated Mediator (Handler type) #6
Re-run all jobs
Summary
All jobs
build-and-test
Run details
Usage
Workflow file
Lockfile
Annotations
1 warning and 1 notice
build-and-test
succeeded yesterday in 22s
Search logs
Set up job 1s
Check out repository 1s
Configure CMake 2s
Build all targets 14s
Run C++ tests 0s
Post Check out repository 0s
Complete job 0s

GOF Gonna Make You Cry...

Home **Content** Live Demo Conclusion





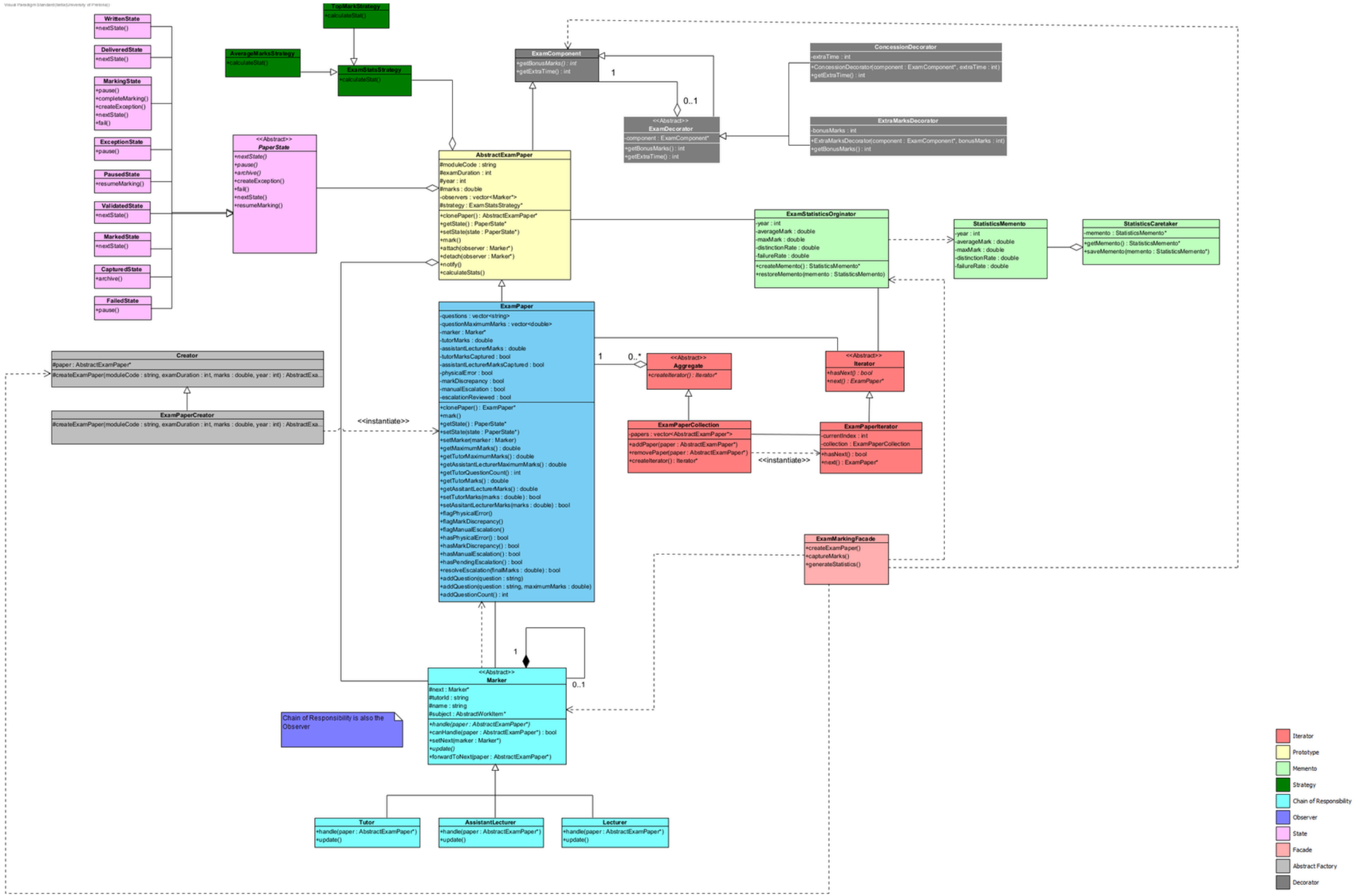


GOF Gonna Say Goodbye...

Home **Content** Live Demo Conclusion

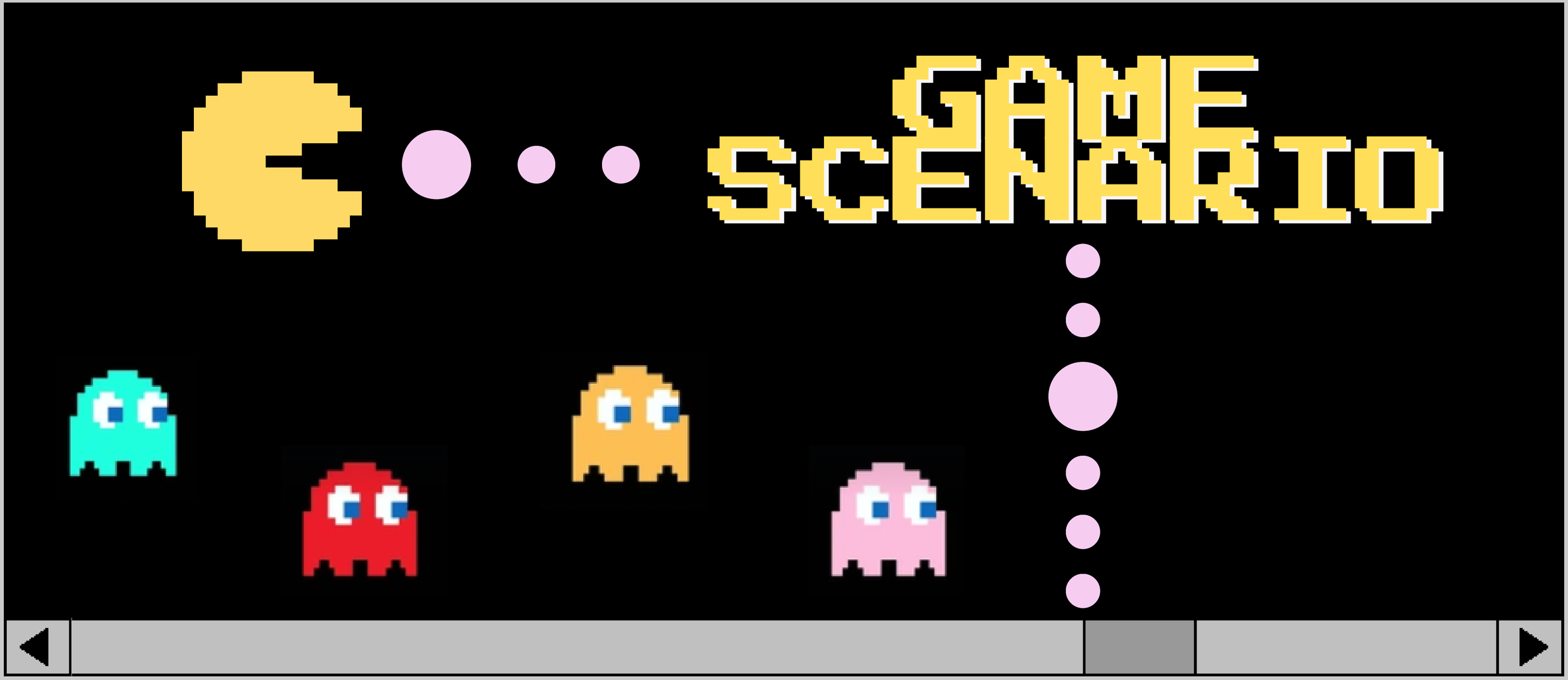
EXAM SCENARIO

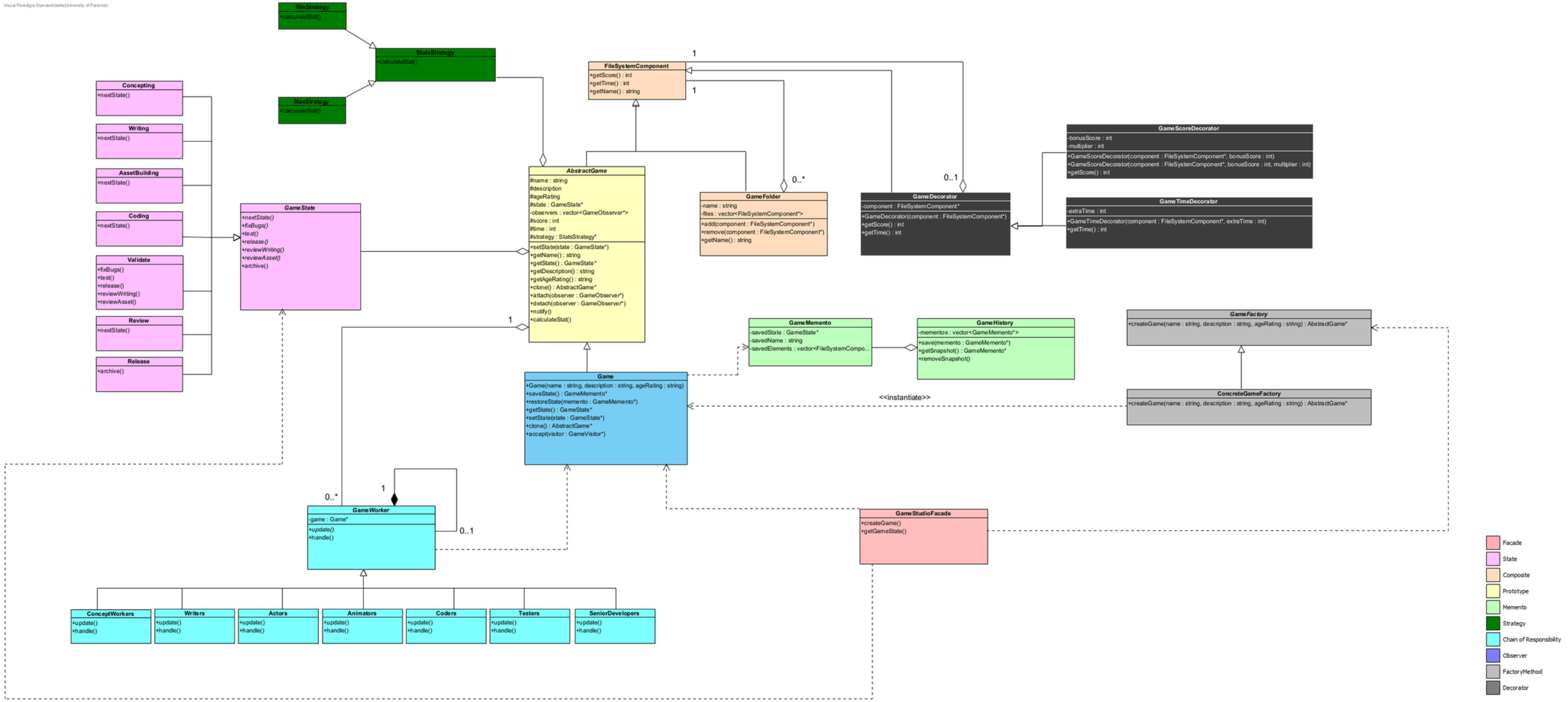



Visual Paradigm Standard(Stella)(University of Pretoria)
WrittenState
+nextState()
DeliveredState
+nextState()
MarkingState
+pause()
+completeMarking()
+createException()
+nextState()
+fail()
ExceptionState
+pause()
PausedState
+resumeMarking()
ValidatedState
+nextState()
MarkedState
+nextState()
CapturedState
+archive()
FailedState
+pause()
TopMarksStrategy
+calculateStat()
AverageMarksStrategy
+calculateStat()
ExamStatsStrategy
+calculateStat()
<<Abstract>>
PaperState
+nextState()
+pause()
+archive()
+createException()
+fail()
+nextState()
+resumeMarking()
ExamComponent
+getBonusMarks() : int
+getExtraTime() : int
1
0..1
ConcessionDecorator
-extraTime : int
+ConcessionDecorator(component : ExamComponent*, extraTime : int)
+getExtraTime() : int
<<Abstract>>
ExamDecorator
-component : ExamComponent*
+getBonusMarks() : int
+getExtraTime() : int
ExtraMarksDecorator
-bonusMarks : int
+ExtraMarksDecorator(component : ExamComponent*, bonusMarks : int)
+getBonusMarks() : int
AbstractExamPaper
#moduleCode : string
#examDuration : int
#year : int
#marks : double
-observers : vector<Marker*>
#strategy : ExamStatsStrategy*
+clonePaper() : AbstractExamPaper*
+getState() : PaperState*
+setState(state : PaperState*)
+mark()
+attach(observer : Marker*)
+detach(observer : Marker*)
+notify()
+calculateStats()
ExamStatisticsOrginator
-year : int
-averageMark : double
-maxMark : double
-distinctionRate : double
-failureRate : double
+createMemento() : StatisticsMemento*
+restoreMemento(memento : StatisticsMemento)
StatisticsMemento
-year : int
-averageMark : double
-maxMark : double
-distinctionRate : double
-failureRate : double
StatisticsCaretaker
-memento : StatisticsMemento*
+getMemento() : StatisticsMemento*
+saveMemento(memento : StatisticsMemento*)
ExamPaper
-questions : vector<string>
-questionMaximumMarks : vector<double>
-marker : Marker*
-tutorMarks : double
-assistantLecturerMarks : double
-tutorMarksCaptured : bool
-assistantLecturerMarksCaptured : bool
-physicalError : bool
-markDiscrepancy : bool
-manualEscalation : bool
-escalationReviewed : bool
+clonePaper() : ExamPaper*
+mark()
+getState() : PaperState*
+setState(state : PaperState*)
+setMarker(marker : Marker)
+getMaximumMarks() : double
+getTutorMaximumMarks() : double
+getAssistantLecturerMaximumMarks() : double
+getTutorQuestionCount() : int
+getTutorMarks() : double
+getAssitantLecturerMarks() : double
+setTutorMarks(marks : double) : bool
+setAssitantLecturerMarks(marks : double) : bool
+flagPhysicalError()
+flagMarkDiscrepancy()
+flagManualEscalation()
+hasPhysicalError() : bool
+hasMarkDiscrepancy() : bool
+hasManualEscalation() : bool
+hasPendingEscalation() : bool
+resolveEscalation(finalMarks : double) : bool
+addQuestion(question : string)
+addQuestion(question : string, maximumMarks : double)
+addQuestionCount() : int
1
0..*
<<Abstract>>
Aggregate
+createIterator() : Iterator*
<<Abstract>>
Iterator
+hasNext() : bool
+next() : ExamPaper*
ExamPaperCollection
-papers : vector<AbstractExamPaper*>
+addPaper(paper : AbstractExamPaper*)
+removePaper(paper : AbstractExamPaper*)
+createIterator() : Iterator*
ExamPaperIterator
-currentIndex : int
-collection : ExamPaperCollection
+hasNext() : bool
+next() : ExamPaper*
<<instantiate>>
Creator
#paper : AbstractExamPaper*
#createExamPaper(moduleCode : string, examDuration : int, marks : double, year : int) : AbstractExa...
ExamPaperCreator
#createExamPaper(moduleCode : string, examDuration : int, marks : double, year : int) : AbstractExa...
<<instantiate>>
ExamMarkingFacade
+createExamPaper()
+captureMarks()
+generateStatistics()
1
0..1
<<Abstract>>
Marker
#next : Marker*
#tutorId : string
#name : string
#subject : AbstractWorkItem*
+handle(paper : AbstractExamPaper*)
+canHandle(paper : AbstractExamPaper*) : bool
+setNext(marker : Marker*)
+update()
+forwardToNext(paper : AbstractExamPaper*)
Chain of Responsibility is also the Observer
Tutor
+handle(paper : AbstractExamPaper*)
+update()
AssistantLecturer
+handle(paper : AbstractExamPaper*)
+update()
Lecturer
+handle(paper : AbstractExamPaper*)
+update()
Iterator
Prototype
Memento
Strategy
Chain of Responsibility
Observer
State
Facade
Abstract Factory
Decorator

GOF Gonna Tell A Lie...

Home **Content** Live Demo Conclusion




Visual Paradigm Standard(bella(University of Pretoria))
MinStrategy
+calculateStat()
StatsStrategy
+calculateStat()
MaxStrategy
+calculateStat()
Concepting
+nextState()
Writing
+nextState()
AssetBuilding
+nextState()
Coding
+nextState()
Validate
+fixBugs()
+test()
+release()
+reviewWriting()
+reviewAsset()
Review
+nextState()
Release
+archive()
GameState
+nextState()
+fixBugs()
+test()
+release()
+reviewWriting()
+reviewAsset()
+archive()
FileSystemComponent
+getScore() : int
+getTime() : int
+getName() : string
AbstractGame
#name : string
#description
#ageRating
#state : GameState*
-observers : vector<GameObserver*>
#score : int
#time : int
#strategy : StatsStrategy*
+setState(state : GameState*)
+getName() : string
+getState() : GameState*
+getDescription() : string
+getAgeRating() : string
+clone() : AbstractGame*
+attach(observer : GameObserver*)
+detach(observer : GameObserver*)
+notify()
+calculateStat()
GameFolder
-name : string
-files : vector<FileSystemComponent*>
+add(component : FileSystemComponent*)
+remove(component : FileSystemComponent*)
+getName() : string
GameDecorator
-component : FileSystemComponent*
+GameDecorator(component : FileSystemComponent*)
+getScore() : int
+getTime() : int
GameScoreDecorator
-bonusScore : int
-multiplier : int
+GameScoreDecorator(component : FileSystemComponent*, bonusScore : int)
+GameScoreDecorator(component : FileSystemComponent*, bonusScore : int, multiplier : int)
+getScore() : int
GameTimeDecorator
-extraTime : int
+GameTimeDecorator(component : FileSystemComponent*, extraTime : int)
+getTime() : int
GameMemento
-savedState : GameState*
-savedName : string
-savedElements : vector<FileSystemCompo...
GameHistory
-mementos : vector<GameMemento*>
+save(memento : GameMemento*)
+getSnapshot() : GameMemento*
+removeSnapshot()
Game
+Game(name : string, description : string, ageRating : string)
+saveState() : GameMemento*
+restoreState(memento : GameMemento*)
+getState() : GameState*
+setState(state : GameState*)
+clone() : AbstractGame*
+accept(visitor : GameVisitor*)
GameFactory
+createGame(name : string, description : string, ageRating : string) : AbstractGame*
ConcreteGameFactory
+createGame(name : string, description : string, ageRating : string) : AbstractGame*
<<instantiate>>
GameWorker
-game : Game*
+update()
+handle()
GameStudioFacade
+createGame()
+getGameState()
ConceptWorkers
+update()
+handle()
Writers
+update()
+handle()
Actors
+update()
+handle()
Animators
+update()
+handle()
Coders
+update()
+handle()
Testers
+update()
+handle()
SeniorDevelopers
+update()
+handle()
1
0..*
0..1
Facade
State
Composite
Prototype
Memento
Strategy
Chain of Responsibility
Observer
FactoryMethod
Decorator

And Hurt You...

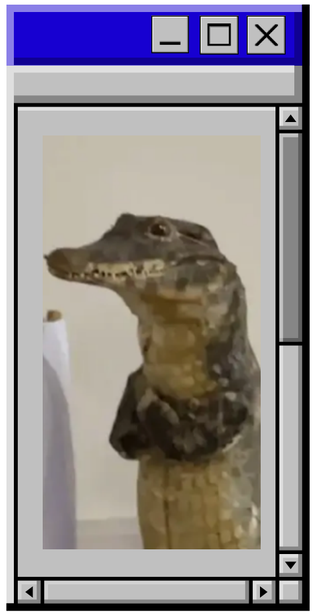
Home **Content** Live Demo Conclusion

DISCLAIMER: AI WAS USED TO ASSIST MAKING OUR FRONTEND



Ooh-ooh (Give you up)...

Home **Content** Live Demo Conclusion

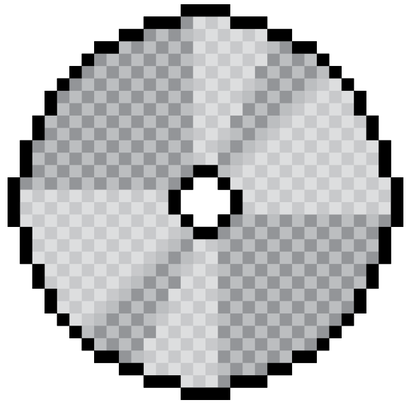


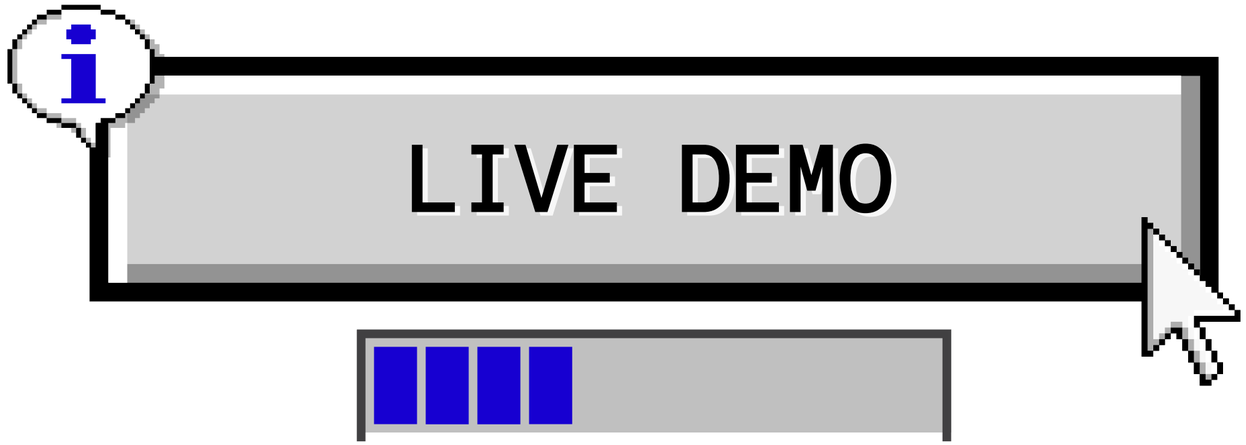




Ooh (Never gonna give, never gonna give)...

Home Content **Live Demo** Conclusion







Give you up.

Home Content Live Demo **Conclusion**

CONCLUSION

http://localhost:8080 @GoF_RickRoll Just sing the song